bonjour

Bonjour ! Comment puis-je vous aider aujourd'hui ?

voici le code du fichier GestionReparation.py : import sqlite3

```
from PyQt6.QtWidgets import QWidget, QMessageBox, QTreeWidgetItem

from PyQt6.uic import loadUi

from PyQt6.QtCore import pyqtSignal, Qt

from datetime import datetime


class GestionReparation(QWidget):


    # SIGNAL Émis lorsque la réparation est terminée (pour GestionVehicule.py)

    reparation_terminee = pyqtSignal()


    def __init__(self, db_connect_func):

        super().__init__()

        # Chargement de l'interface utilisateur

        loadUi("GestionReparation.ui", self)


        self.db_connect = db_connect_func
```

```
    # Initialisation de la date à aujourd'hui
    self.date_debut.setDate(datetime.now().date())
    self.date_fin.setDate(datetime.now().date()) # Date de fin par défaut = date de début

    # Connexion des boutons
    self.btn_enregistrer.clicked.connect(self.enregistrer_reparation)
    self.btn_cloturer.clicked.connect(self.cloturer_reparation)

    # Connexion de l'événement de sélection pour remplir les champs
    self.tableReparations.itemSelectionChanged.connect(self.charger_champs_reparation)

    # Initialisation des données
    self.charger_vehicules_a_reparer()
    self.load_reparations()

# --- Fonctions de chargement et d'affichage ---

def charger_vehicules_a_reparer(self):
```

```
"""Charge les listes déroulantes (véhicules et fournisseurs)."""
try:
    conn = self.db_connect()
    cursor = conn.cursor()

    # 1. Véhicules
    cursor.execute("SELECT immatriculation, statut_disponibilite FROM VEHICULE ORDER BY immatriculation")
    vehicules_data = cursor.fetchall()

    self.combo_vehicule.clear()
    self.combo_vehicule.addItem("Sélectionner un véhicule...", "") # Option par défaut (data = "")

    # Affichage de l'immatriculation et du statut (Immat stockée dans UserData)
    for immat, statut in vehicules_data:
        # Stocker l'immatriculation comme userData pour la récupérer facilement
        self.combo_vehicule.addItem(f"{immat} ({statut})", immat)

    # 2. Fournisseurs (Chargement statique pour la démo)
    if self.combo_fournisseur.count() <= 1:
```

```
            self.combo_fournisseur.clear()
            self.combo_fournisseur.addItem("Choisir un fournisseur...", "")
            self.combo_fournisseur.addItems(['Garage Interne', 'Fournisseur A', 'Fournisseur B'])



        except sqlite3.Error as e:
            QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des ComboBoxes: {e}")
        finally:
            if 'conn' in locals() and conn:
                conn.close()

    def load_reparations(self):
        """Charge toutes les réparations dans le QTreeWidget."""
        try:
            conn = self.db_connect()
            cursor = conn.cursor()

            # Le 7ème élément est Cout MO. On suppose que le 6ème est Statut
            cursor.execute("""
```

```
    SELECT
        id_reparation, immatriculation, type_maintenance,
        nature_panne_tache, date_debut_reparation, statut_reparation,
        COALESCE(cout_main_d_oeuvre, 0)
    FROM REPARATION
    ORDER BY id_reparation DESC
""")
reparations = cursor.fetchall()

self.tableReparations.clear()

# Ajustement des labels pour coller aux colonnes récupérées
self.tableReparations.setHeaderLabels([
    "ID Rép.", "Véhicule", "Type", "Nature",
    "Début", "Statut", "Coût MO"
])

for row_data in reparations:
    item = QTreeWidgetItem(self.tableReparations)
```

```
            id_reparation = row_data[0]

            for col_num, data in enumerate(row_data):
                # Formatage du Coût MO (colonne 6)
                display_data = f"{data:,.2f} €" if col_num == 6 and data is not None else str(data if data is not None else '')
                item.setText(col_num, display_data)

            # Stocker l'ID de la réparation pour la clôture
            item.setData(0, Qt.ItemDataRole.UserRole, id_reparation)

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des réparations: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()

def charger_champs_reparation(self):
    """Remplit les champs avec les données de la réparation sélectionnée (pour la clôture)."""
    selected_item = self.tableReparations.currentItem()
```

```
    if selected_item:
        # Récupérer le coût (colonne 6) et supprimer le formatage ' €'
        # Nécessaire pour éviter une erreur ValueError lors de la reconversion en float si l'utilisateur modifie
        cout_mo_text = selected_item.text(6).replace(' €', '').replace('.', '').replace(',', '.')
        self.input_cout_mo.setText(cout_mo_text)

# --- Fonctions d'Action ---

def valider_cout_mo(self, cout_mo_str):
    """Nettoie et convertit la chaîne du coût en float, ou None si vide/erreur."""
    if not cout_mo_str:
        return None # Autoriser NULL dans la DB si non renseigné

    try:
        # Remplacement de la virgule par un point et conversion en float
        cout_mo_nettoye = cout_mo_str.replace(',', '.')
        cout_mo = float(cout_mo_nettoye)

        if cout_mo < 0:
```

```
            QMessageBox.warning(self, "Erreur de Saisie", "Le coût de main-d'œuvre ne peut pas être négatif.")
            return False
        return cout_mo

    except ValueError:
        QMessageBox.critical(self, "Erreur de Format", "Le coût de main-d'œuvre doit être un nombre valide.")
        return False

def enregistrer_reparation(self):
    """Enregistre une nouvelle intervention (REPARATION) et met le véhicule 'En réparation'."""

    immatriculation = self.combo_vehicule.currentData()
    type_maintenance = self.combo_type_maintenance.currentText().split('(')[0].strip()
    nature = self.input_nature.text().strip()
    date_debut = self.date_debut.date().toString("yyyy-MM-dd")

    # Récupération et validation/conversion du Coût MO (CORRECTION ici)
    cout_mo_valide = self.valider_cout_mo(self.input_cout_mo.text().strip())
    if cout_mo_valide is False:
```

```
    return
cout_mo = cout_mo_valide # Sera un float ou None


if not all([immatriculation, nature, date_debut]):
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez remplir le véhicule, la nature et la date de début.")
    return


try:
    conn = self.db_connect()
    cursor = conn.cursor()


    # 1. Insérer la réparation (MODIFICATION de la requête SQL)
    cursor.execute("""
        INSERT INTO REPARATION (immatriculation, type_maintenance, nature_panne_tache,
                                date_debut_reparation, statut_reparation, cout_main_d_oeuvre)
        VALUES (?, ?, ?, ?, 'En cours', ?)
    """, (immatriculation, type_maintenance, nature, date_debut, cout_mo)) # cout_mo est inséré ici


    # 2. Mettre le statut du VEHICULE à 'En réparation'
```

```
        cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'En réparation' WHERE immatriculation = ?", (immatriculation,))


        conn.commit()
        QMessageBox.information(self, "Succès", f"Réparation enregistrée pour {immatriculation}. Statut du véhicule mis à jour.")
        self.load_reparations()
        self.charger_vehicules_a_reparer()
        self.input_cout_mo.clear() # Nettoyer le champ après l'enregistrement


    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible d'enregistrer la réparation: {e}")
        if 'conn' in locals() and conn:
            conn.rollback()
    finally:
        if 'conn' in locals() and conn:
            conn.close()



# def cloturer_reparation(self):
#     """Clôture l'intervention sélectionnée, met à jour le coût/date et remet le véhicule 'Disponible'."""
```

```
#    selected_item = self.tableReparations.currentItem()
#    if not selected_item:
#        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner une réparation 'En cours' à clôturer.")
#        return

#    id_reparation = selected_item.data(0, Qt.ItemDataRole.UserRole)
#    immatriculation = selected_item.text(1)
#    statut_actuel = selected_item.text(5)

#    if statut_actuel != 'En cours':
#        QMessageBox.warning(self, "Statut Invalide", "Seules les interventions 'En cours' peuvent être clôturées.")
#        return

#    # Récupération et validation/conversion du Coût MO
#    cout_mo_valide = self.valider_cout_mo(self.input_cout_mo.text().strip())
#    if cout_mo_valide is False:
#        return
#    cout_mo = cout_mo_valide # Sera un float ou None
```

```
#    date_fin = self.date_fin.date().toString("yyyy-MM-dd")

#    try:
#        conn = self.db_connect()
#        cursor = conn.cursor()

#        # 1. Mettre à jour la réparation
#        cursor.execute("""
#            UPDATE REPARATION SET
#            statut_reparation = 'Terminée',
#            date_fin_reparation = ?,
#            cout_main_d_oeuvre = ?
#            WHERE id_reparation = ?
#        """, (date_fin, cout_mo, id_reparation))

#        # 2. Mettre à jour le statut du VEHICULE à 'Disponible'
#        cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'Disponible' WHERE immatriculation = ?", (immatriculation,))

#        conn.commit()
```

```
#        QMessageBox.information(self, "Succès", f"Réparation ID {id_reparation} clôturée. Le véhicule est de nouveau disponible.")
#        self.load_reparations()
#        self.charger_vehicules_a_reparer()
#        self.reparation_terminee.emit() # Émission du signal pour synchronisation

#    except sqlite3.Error as e:
#        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de clôturer la réparation: {e}")
#        if 'conn' in locals() and conn:
#            conn.rollback()
#    finally:
#        if 'conn' in locals() and conn:
#            conn.close()


# CODE DANS GestionReparation.py (Seule la fonction cloturer_reparation est modifiée)

# ... (Le reste des fonctions reste identique)

    def cloturer_reparation(self):
```

```
"""Clôture l'intervention sélectionnée, met à jour le coût/date, description des travaux et remet le véhicule 'Disponible'."""
selected_item = self.tableReparations.currentItem()
if not selected_item:
    QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner une réparation 'En cours' à clôturer.")
    return

id_reparation = selected_item.data(0, Qt.ItemDataRole.UserRole)
immatriculation = selected_item.text(1)
statut_actuel = selected_item.text(5)

if statut_actuel != 'En cours':
    QMessageBox.warning(self, "Statut Invalide", "Seules les interventions 'En cours' peuvent être clôturées.")
    return

# Récupération et validation/conversion du Coût MO
cout_mo_valide = self.valider_cout_mo(self.input_cout_mo.text().strip())
if cout_mo_valide is False:
     return
cout_mo = cout_mo_valide # Sera un float ou None
```

```
date_fin = self.date_fin.date().toString("yyyy-MM-dd")

# --- CORRECTION : Récupération de la description des travaux ---
# Utilisation de .toPlainText() si c'est un QTextEdit, ou .text() si c'est un QLineEdit
description_travaux = self.input_description_travaux.toPlainText().strip()

if not description_travaux:
    QMessageBox.warning(self, "Saisie Requise", "Veuillez saisir la description des travaux effectués (Description Travaux).")
    return

try:
    conn = self.db_connect()
    cursor = conn.cursor()

    # 1. Mettre à jour la réparation (INCLUSION de description_travaux)
    cursor.execute("""
        UPDATE REPARATION SET
        statut_reparation = 'Terminée',
```

```
        date_fin_reparation = ?,
        cout_main_d_oeuvre = ?,
        description_travaux = ?
        WHERE id_reparation = ?
    """, (date_fin, cout_mo, description_travaux, id_reparation))

    # 2. Mettre à jour le statut du VEHICULE à 'Disponible'
    cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'Disponible' WHERE immatriculation = ?", (immatriculation,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Réparation ID {id_reparation} clôturée. Le véhicule est de nouveau disponible.")
    self.load_reparations()
    self.charger_vehicules_a_reparer()
    self.reparation_terminee.emit()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de clôturer la réparation: {e}")
    if 'conn' in locals() and conn:
        conn.rollback()
```

```
    finally:
        if 'conn' in locals() and conn:
            conn.close()


# ... (Reste du code)
```

C'est le code Python pour la classe `GestionReparation` qui utilise **PyQt6** et **SQLite3** pour gérer les réparations de véhicules.

Cette version du code est une classe complète, et la fonction `cloturer_reparation` a été **modifiée** et **commentée** pour inclure la gestion du champ `description_travaux`.

Résumé des Fonctionnalités Principales

Section	Description
`__init__`	Initialise l'interface utilisateur (via `GestionReparation.ui`), se connecte à la base de données via une fonction passée en paramètre, initialise les dates, et connecte les boutons (`btn_enregistrer`, `btn_cloturer`) et la sélection du tableau.
`reparation_terminee`	Un **signal PyQt** émis lorsqu'une réparation est clôturée, pour synchronisation avec d'autres modules (ex: `GestionVehicule`).
`charger_vehicules_a_reparer`	Charge la liste des véhicules et leur statut dans la `combo_vehicule`, et une liste statique de fournisseurs.
`load_reparations`	Récupère et affiche toutes les réparations dans le **`tableReparations`** (`QTreeWidget`), en incluant l'ID, l'immatriculation, le statut, la date et le coût de la main-d'œuvre.
`charger_champs_reparation`	Lorsque l'utilisateur sélectionne une ligne dans le tableau, cette fonction remplit le champ de coût de main-d'œuvre (`input_cout_mo`) pour faciliter la clôture.
`valider_cout_mo`	Valide la saisie du coût de main-d'œuvre (format numérique, non-négatif) et le convertit en `float` (ou `None` si vide).
`enregistrer_reparation`	1. **Insère** une nouvelle entrée dans la table `REPARATION` avec le statut `'En cours'`. 2. **Met à jour** le statut du `VEHICULE` correspondant à `'En réparation'`.

Section	Description
`cloturer_reparation`	1. **Vérifie** que la réparation est `'En cours'` et qu'une description des travaux a été fournie. 2. **Met à jour** l'entrée dans `REPARATION` en définissant `statut_reparation = 'Terminée'`, `date_fin_reparation`, `cout_main_d_oeuvre` et **`description_travaux`**. 3. **Met à jour** le statut du `VEHICULE` à `'Disponible'`. 4. **Émet** le signal `reparation_terminee`.

Modifications Notables dans `cloturer_reparation`

La fonction `cloturer_reparation` a été enrichie pour prendre en compte le champ **`description_travaux`**, qui est supposé être un widget de type `QTextEdit` ou similaire dans le fichier `.ui` (d'où l'utilisation de `.toPlainText()`):

1. **Récupération du champ:**

   Python

- `description_travaux = self.input_description_travaux.toPlainText().strip()`

- **Validation de l'entrée:**

Python
-
```
if not description_travaux:
 QMessageBox.warning(self, "Saisie Requise", "Veuillez saisir la description des travaux effectués (Description Travaux).")
 return
```

- **Mise à jour SQL:** Le champ `description_travaux` est inclus dans la requête `UPDATE`:

SQL
```
UPDATE REPARATION SET
statut_reparation = 'Terminée',
date_fin_reparation = ?,
cout_main_d_oeuvre = ?,
description_travaux = ?  -- Nouveau champ ajouté ici
WHERE id_reparation = ?
```

et dans les paramètres d'exécution:

Python
```
3. """, (date_fin, cout_mo, description_travaux, id_reparation))
```

4.

code du fichier GestionReparation.ui : <?xml version="1.0" encoding="UTF-8"?>

```
<ui version="4.0">
 <class>GestionReparation</class>
 <widget class="QWidget" name="GestionReparation">
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>750</width>
    <height>650</height> </rect>
  </property>
  <property name="windowTitle">
   <string>Gestion des Réparations et Maintenance</string>
  </property>
  <layout class="QVBoxLayout" name="mainLayout">
   <item>
    <widget class="QLabel" name="label_titre">
     <property name="styleSheet">
```

```
   <string notr="true">font: 18pt "Times New Roman"; color: rgb(0, 85, 127);</string>
  </property>
  <property name="text">
   <string>Suivi des Réparations et Maintenance</string>
  </property>
  <property name="alignment">
   <set>Qt::AlignCenter</set>
  </property>
 </widget>
</item>
<item>
 <widget class="QGroupBox" name="formGroup">
  <property name="title">
   <string>Détails de l'Intervention</string>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="label_vehicule">
     <property name="text">
```

```
   <string>&amp;amp;Véhicule (Immat.) :</string>
  </property>
  <property name="buddy">
   <cstring>combo_vehicule</cstring>
  </property>
 </widget>
</item>
<item row="0" column="1">
 <widget class="QComboBox" name="combo_vehicule"/>
</item>


<item row="1" column="0">
 <widget class="QLabel" name="label_type_maintenance">
  <property name="text">
   <string>&amp;amp;Type de Maintenance :</string>
  </property>
  <property name="buddy">
   <cstring>combo_type_maintenance</cstring>
  </property>
```

```
 </widget>
</item>
<item row="1" column="1">
 <widget class="QComboBox" name="combo_type_maintenance">
  <item>
   <property name="text">
    <string>Corrective (Panne)</string>
   </property>
  </item>
  <item>
   <property name="text">
    <string>Préventive (Planifiée)</string>
   </property>
  </item>
 </widget>
</item>

<item row="2" column="0">
 <widget class="QLabel" name="label_panne">
```

```
  <property name="text">
   <string>&amp;amp;Nature de la Panne/Tâche :</string>
  </property>
  <property name="buddy">
   <cstring>input_nature</cstring>
  </property>
 </widget>
</item>
<item row="2" column="1">
 <widget class="QLineEdit" name="input_nature"/>
</item>

<item row="3" column="0">
 <widget class="QLabel" name="label_date_debut">
  <property name="text">
   <string>Date &amp;amp;Début :</string>
  </property>
  <property name="buddy">
   <cstring>date_debut</cstring>
```

```
  </property>
 </widget>
</item>
<item row="3" column="1">
 <widget class="QDateEdit" name="date_debut"/>
</item>


<item row="4" column="0">
 <widget class="QLabel" name="label_fournisseur">
  <property name="text">
   <string>&amp;amp;Garage/Fournisseur :</string>
  </property>
  <property name="buddy">
   <cstring>combo_fournisseur</cstring>
  </property>
 </widget>
</item>
<item row="4" column="1">
 <widget class="QComboBox" name="combo_fournisseur"/>
```

```
</item>

<item row="5" column="0">
 <widget class="QLabel" name="label_cout_mo">
  <property name="text">
   <string>Coût Main-d'œuvre (HT) :</string>
  </property>
  <property name="buddy">
   <cstring>input_cout_mo</cstring>
  </property>
 </widget>
</item>
<item row="5" column="1">
 <widget class="QLineEdit" name="input_cout_mo"/>
</item>

<item row="6" column="0">
 <widget class="QLabel" name="label_description_travaux">
  <property name="text">
```

```
   <string>D&amp;amp;escription Travaux :</string>
  </property>
  <property name="buddy">
   <cstring>input_description_travaux</cstring>
  </property>
 </widget>
</item>
<item row="6" column="1">
 <widget class="QTextEdit" name="input_description_travaux">
     <property name="minimumHeight">
         <number>60</number>
     </property>
 </widget>
</item>


<item row="7" column="0">
 <widget class="QLabel" name="label_date_fin">
  <property name="text">
   <string>Date &amp;amp;Fin :</string>
```

```
    </property>
    <property name="buddy">
     <cstring>date_fin</cstring>
    </property>
   </widget>
  </item>
  <item row="7" column="1">
   <widget class="QDateEdit" name="date_fin"/>
  </item>


 </layout>
</widget>
</item>
<item>
 <layout class="QHBoxLayout" name="buttonLayout">
  <item>
   <spacer name="horizontalSpacer_left">
    <property name="orientation">
     <enum>Qt::Horizontal</enum>
```

```
  </property>
  <property name="sizeHint" stdset="0">
   <size>
    <width>40</width>
    <height>20</height>
   </size>
  </property>
 </spacer>
</item>
<item>
 <widget class="QPushButton" name="btn_enregistrer">
  <property name="text">
   <string>Enregistrer Réparation</string>
  </property>
 </widget>
</item>
<item>
 <widget class="QPushButton" name="btn_cloturer">
  <property name="text">
```

```
    <string>Clôturer et Facturer</string>
   </property>
  </widget>
 </item>
 <item>
  <spacer name="horizontalSpacer_right">
   <property name="orientation">
    <enum>Qt::Horizontal</enum>
   </property>
   <property name="sizeHint" stdset="0">
    <size>
     <width>40</width>
     <height>20</height>
    </size>
   </property>
  </spacer>
 </item>
</layout>
</item>
```

```
<item>
 <widget class="QTreeWidget" name="tableReparations">
  <column>
   <property name="text">
    <string>ID Rép.</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Véhicule</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Type</string>
   </property>
  </column>
  <column>
   <property name="text">
```

```
      <string>Nature</string>
     </property>
    </column>
    <column>
     <property name="text">
      <string>Statut</string>
     </property>
    </column>
    <column>
     <property name="text">
      <string>Coût MO</string>
     </property>
    </column>
    </widget>
  </item>
 </layout>
</widget>
<resources/>
<connections/>
```

</ui>

Le code que vous avez fourni est le fichier **XML** généré par **Qt Designer** pour définir l'interface utilisateur graphique (**GUI**) de la fenêtre `GestionReparation`.

Ce fichier, nommé `GestionReparation.ui`, est chargé par le code Python que vous avez montré précédemment (`GestionReparation.py`) via la ligne `loadUi("GestionReparation.ui", self)`.

Structure de l'Interface (`GestionReparation.ui`)

L'interface est structurée autour d'un **`QVBoxLayout`** (mise en page verticale principale) et se compose de trois sections principales :

1. Titre

- **`QLabel`** nommé **`label_titre`** : Affiche le titre centré "Suivi des Réparations et Maintenance" avec un style de police spécifique.

---

2. Formulaire de Détails de l'Intervention (`QGroupBox`)

Ce groupe est organisé via un **`QFormLayout`**, présentant les étiquettes (labels) à gauche et les widgets de saisie à droite. Il contient les éléments suivants :

Label (Étiquette)	Widget (Type / Nom)	Description	Utilisé dans `GestionReparation.py`
Véhicule (Immat.)	**`QComboBox`** (`combo_vehicule`)	Liste déroulante pour sélectionner le véhicule à réparer (Immatriculation).	Pour l'enregistrement de la réparation.
Type de Maintenance	**`QComboBox`** (`combo_type_maintenance`)	Liste déroulante pré-remplie avec "Corrective" et "Préventive".	Pour l'enregistrement de la réparation.
Nature de la Panne/Tâche	**`QLineEdit`** (`input_nature`)	Champ de texte pour saisir la description initiale de la panne ou de la tâche.	Pour l'enregistrement de la réparation.
Date Début	**`QDateEdit`** (`date_debut`)	Sélecteur de date pour le début de la réparation.	Pour l'enregistrement de la réparation.
Garage/Fournisseur	**`QComboBox`** (`combo_fournisseur`)	Liste déroulante pour sélectionner le fournisseur (garage).	Pour l'enregistrement (utilisé mais non persistant dans l'exemple DB).
Coût Main-d'œuvre (HT)	**`QLineEdit`** (`input_cout_mo`)	Champ de texte pour la saisie du coût de la main-d'œuvre.	Pour l'enregistrement *et* la clôture de la réparation.

Label (Étiquette)	Widget (Type / Nom)	Description	Utilisé dans `GestionReparation.py`
Description Travaux	**`QTextEdit`** (`input_description_travaux`)	Zone de texte pour les travaux effectués. **Important pour la clôture.**	Utilisé dans la fonction **`cloturer_reparation`**.
Date Fin	**`QDateEdit`** (`date_fin`)	Sélecteur de date pour la date de clôture de la réparation.	Utilisé dans la fonction **`cloturer_reparation`**.

---

3. Boutons d'Action et Tableau

- **Boutons d'Action (`QHBoxLayout`)** :
  - **`QPushButton`** nommé **`btn_enregistrer`** : Connecté à la méthode `enregistrer_reparation()`.
  - **`QPushButton`** nommé **`btn_cloturer`** : Connecté à la méthode `cloturer_reparation()`.
- **Tableau d'Affichage (`QTreeWidget`)** :
  - Nommé **`tableReparations`**.
  - C'est là que la méthode `load_reparations()` affiche la liste des interventions.
  - Les colonnes prédéfinies sont : "ID Rép.", "Véhicule", "Type", "Nature", "Statut", "Coût MO".

Note sur la synchronisation : Le widget **`input_description_travaux`** est un **`QTextEdit`** (permettant la saisie multiligne), ce qui correspond à l'utilisation de **`.toPlainText()`** dans votre fonction Python `cloturer_reparation()`.

code du fichier GestionStockEtCommandes.py :

```
import sqlite3

from PyQt6.QtWidgets import QWidget, QMessageBox, QTableWidgetItem

from PyQt6.uic import loadUi

from PyQt6.QtCore import QDate



# 4. Fichier : GestionStockEtCommandes.py
```

```
# Ce fichier gère l'interface complexe des pièces, du stock et des commandes (Tables: PIECE, FOURNISSEUR, COMMANDE_REPARATION, UTILISER).


class GestionStockEtCommandes(QWidget):


    def __init__(self, db_connect_func):
        super().__init__()
        # Chargement de l'interface utilisateur
        loadUi("GestionStockEtCommandes.ui", self)


        self.db_connect = db_connect_func


        # Connexion des boutons
        self.btn_ajouter_piece.clicked.connect(self.ajouter_piece)
        self.btn_passer_commande.clicked.connect(self.passer_commande_reparation)
        self.btn_recevoir_commande.clicked.connect(self.recevoir_commande)


        # Initialisation des données
        self.charger_pieces()
        self.charger_fournisseurs(self.combo_fournisseur_commande)
```

```
    self.charger_commandes()


# --- Fonctions utilitaires ---
def charger_fournisseurs(self, combobox):
    """Charge les fournisseurs (FOURNISSEUR) dans un QComboBox."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()
        cursor.execute("SELECT id_fournisseur, nom_fournisseur FROM FOURNISSEUR")
        fournisseurs = cursor.fetchall()


        combobox.clear()
        for id_f, nom in fournisseurs:
            combobox.addItem(f"{id_f} - {nom}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()


# --- GESTION DES PIECES (PIECE) ---
```

```
def charger_pieces(self):
    """Charge toutes les pièces en stock dans le tableau dédié."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()
        cursor.execute("SELECT id_piece, nom_piece, reference_fournisseur, prix_unitaire, quantite_stock FROM PIECE")
        pieces = cursor.fetchall()

        self.table_pieces.setRowCount(len(pieces))
        for row_num, row_data in enumerate(pieces):
            for col_num, data in enumerate(row_data):
                self.table_pieces.setItem(row_num, col_num, QTableWidgetItem(str(data)))

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des pièces: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()
```

```
def ajouter_piece(self):
    """Ajoute une nouvelle pièce au stock (PIECE)."""
    nom = self.input_nom_piece.text().strip()
    ref = self.input_ref_piece.text().strip()
    prix_unitaire = self.input_prix_unitaire.text().strip().replace(',', '.')
    quantite = self.input_quantite_stock.text().strip()

    try:
        prix = float(prix_unitaire)
        stock = int(quantite)

        conn = self.db_connect()
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO PIECE (nom_piece, reference_fournisseur, prix_unitaire, quantite_stock)
            VALUES (?, ?, ?, ?)
        """, (nom, ref, prix, stock))
```

```
        conn.commit()
        QMessageBox.information(self, "Succès", f"Pièce '{nom}' ajoutée au stock.")
        self.charger_pieces()

    except ValueError:
        QMessageBox.critical(self, "Erreur de Format", "Le prix et la quantité doivent être des nombres valides.")
    except sqlite3.IntegrityError:
        QMessageBox.critical(self, "Erreur SQL", "La référence fournisseur est déjà utilisée.")
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Impossible d'ajouter la pièce: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()

# --- GESTION DES COMMANDES (COMMANDE_REPARATION, UTILISER) ---
def charger_commandes(self):
    """Charge les commandes en cours (COMMANDE_REPARATION) dans le tableau."""
    try:
        conn = self.db_connect()
```

```
cursor = conn.cursor()


cursor.execute("""
    SELECT
        C.id_commande,
        F.nom_fournisseur,
        C.date_commande,
        C.montant_estime,
        C.statut_commande
    FROM COMMANDE_REPARATION C
    JOIN FOURNISSEUR F ON C.id_fournisseur = F.id_fournisseur
    ORDER BY C.date_commande DESC
""")
commandes = cursor.fetchall()


self.table_commandes.setRowCount(len(commandes))
for row_num, row_data in enumerate(commandes):
    for col_num, data in enumerate(row_data):
        self.table_commandes.setItem(row_num, col_num, QTableWidgetItem(str(data)))
```

```
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des commandes: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()



def passer_commande_reparation(self):
    """Crée une nouvelle commande (COMMANDE_REPARATION)."""
    # Dans un système réel, ceci devrait ouvrir une boîte de dialogue pour sélectionner les pièces et les quantités
    try:
        id_fournisseur = int(self.combo_fournisseur_commande.currentText().split(" - ")[0])
        montant_estime = float(self.input_montant_commande.text().strip().replace(',', '.') or 0.0)

        conn = self.db_connect()
        cursor = conn.cursor()

        # 1. Insertion dans COMMANDE_REPARATION
```

```
date_commande = QDate.currentDate().toString("yyyy-MM-dd")
cursor.execute("""
    INSERT INTO COMMANDE_REPARATION (date_commande, montant_estime, id_fournisseur, statut_commande)
    VALUES (?, ?, ?, ?)
""", (date_commande, montant_estime, id_fournisseur, 'En cours'))

id_commande = cursor.lastrowid

# Note: Ici, nous ajoutons une pièce factice pour que la table UTILISER ne soit pas vide
# et que la fonction recevoir_commande fonctionne.
# Remplacer par la logique UI réelle pour sélectionner les pièces.
cursor.execute("SELECT id_piece FROM PIECE LIMIT 1")
first_piece = cursor.fetchone()
if first_piece:
    cursor.execute("""
        INSERT INTO UTILISER (id_commande, id_piece, quantite_utilisee)
        VALUES (?, ?, ?)
    """, (id_commande, first_piece[0], 5)) # Commandé 5 unités d'une pièce existante
```

```
        # Note 2: Si la commande est pour une REPARATION spécifique, il faudrait insérer dans PASSER ici.


        conn.commit()
        QMessageBox.information(self, "Succès", f"Commande ID {id_commande} passée avec succès.")
        self.charger_commandes()


    except IndexError:
        QMessageBox.critical(self, "Erreur de Saisie", "Veuillez sélectionner un fournisseur.")
    except ValueError:
        QMessageBox.critical(self, "Erreur de Format", "Le montant doit être un nombre valide.")
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Impossible de passer la commande: {e}")
        conn.rollback()
    finally:
        if 'conn' in locals() and conn:
            conn.close()



def recevoir_commande(self):
```

```
"""Met à jour le statut de la commande à 'Livrée' et met à jour le stock de pièces (PIECE, COMMANDE_REPARATION, UTILISER)."""
selected_items = self.table_commandes.selectedItems()
if not selected_items:
    QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner une commande à recevoir.")
    return

id_commande = int(selected_items[0].text())
statut_actuel = selected_items[0].text(4)

if statut_actuel == 'Livrée':
    QMessageBox.information(self, "Déjà Livrée", "Cette commande est déjà marquée comme livrée.")
    return

try:
    conn = self.db_connect()
    cursor = conn.cursor()

    # 1. Récupérer les détails des pièces de la commande (UTILISER)
    cursor.execute("""
```

```
    SELECT id_piece, quantite_utilisee FROM UTILISER WHERE id_commande = ?
""", (id_commande,))
pieces_a_recevoir = cursor.fetchall()

if not pieces_a_recevoir:
    QMessageBox.warning(self, "Commande Vide", "Cette commande ne contient aucune pièce à recevoir (vérifiez la table UTILISER).")
    return

# 2. Mettre à jour la quantité en stock pour chaque pièce (PIECE)
for id_piece, quantite in pieces_a_recevoir:
    cursor.execute("""
        UPDATE PIECE SET quantite_stock = quantite_stock + ? WHERE id_piece = ?
    """, (quantite, id_piece))


# 3. Mettre à jour le statut de la commande (COMMANDE_REPARATION)
cursor.execute("""
    UPDATE COMMANDE_REPARATION SET statut_commande = 'Livrée' WHERE id_commande = ?
```

```
        """, (id_commande,))

        conn.commit()
        QMessageBox.information(self, "Succès", f"Commande ID {id_commande} reçue. Stock mis à jour.")
        self.charger_commandes()
        self.charger_pieces()

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Impossible de recevoir la commande et mettre à jour le stock: {e}")
        conn.rollback()
    finally:
        if 'conn' in locals() and conn:
            conn.close()
```

Le code que vous avez fourni est le fichier Python `GestionStockEtCommandes.py`. C'est une classe **PyQt6** qui gère l'interface et la logique métier pour les pièces de rechange, le stock, les fournisseurs, et les commandes, interagissant avec les tables de base de données **`PIECE`**, **`FOURNISSEUR`**, **`COMMANDE_REPARATION`**, et **`UTILISER`**.

Résumé des Fonctionnalités Clés

Ce module divise ses responsabilités en deux grandes catégories : la gestion des pièces de stock et la gestion des commandes de réparation.

1. Initialisation et Utilitaires

- **`__init__`**: Charge l'interface utilisateur (`GestionStockEtCommandes.ui`), connecte les méthodes aux boutons (**Ajouter Pièce**, **Passer Commande**, **Recevoir Commande**), et initialise les données dans l'interface.
- **`charger_fournisseurs(combobox)`** : Récupère les `id_fournisseur` et `nom_fournisseur` de la table **`FOURNISSEUR`** et les affiche dans un `QComboBox` donné.

---

2. Gestion des Pièces (Stock)

Cette section gère l'ajout de nouvelles pièces et l'affichage de l'état du stock (table **`PIECE`**).

- **`charger_pieces()`**:
  - Récupère l'ID, le nom, la référence, le prix unitaire et la quantité en stock de toutes les pièces.
  - Affiche ces données dans le tableau **`table_pieces`**.
- **`ajouter_piece()`**:
  - Récupère le nom, la référence, le prix unitaire et la quantité du stock depuis les champs de saisie (`QLineEdit`).
  - Valide que le prix et la quantité sont des nombres.
  - **Insère** une nouvelle entrée dans la table **`PIECE`**.
  - Met à jour l'affichage du stock.

---

3. Gestion des Commandes

Cette section gère le cycle de vie des commandes passées aux fournisseurs (tables **`COMMANDE_REPARATION`** et **`UTILISER`**).

- **`charger_commandes()`**:
  - Récupère l'ID de la commande, le nom du fournisseur, la date, le montant estimé et le statut (des tables **`COMMANDE_REPARATION`** et **`FOURNISSEUR`**).
  - Affiche les commandes dans le tableau **`table_commandes`**.
- **`passer_commande_reparation()`**:
  - Récupère l'ID du fournisseur sélectionné et le montant estimé.
  - **Insère** une nouvelle commande dans la table **`COMMANDE_REPARATION`** avec le statut `'En cours'`.
  - **Simule** l'ajout d'une pièce à la commande dans la table d'association **`UTILISER`** pour que la fonction de réception fonctionne correctement (cette partie est notée comme une implémentation simplifiée qui devrait être remplacée par une logique UI complète).
- **`recevoir_commande()`**:
  - Vérifie qu'une commande `'En cours'` est sélectionnée.

- **Récupère** les détails des pièces et quantités commandées via la table **`UTILISER`**.
- Pour chaque pièce commandée, **met à jour** la `quantite_stock` dans la table **`PIECE`** en ajoutant les quantités reçues.
- **Met à jour** le `statut_commande` de la commande à `'Livrée'` dans la table **`COMMANDE_REPARATION`**.
- Actualise les affichages des commandes et du stock de pièces.

code du fichier GestionStockEtCommandes.ui : <?xml version="1.0" encoding="UTF-8"?>

```
<ui version="4.0">

 <class>GestionStockEtCommandes</class>

 <widget class="QWidget" name="GestionStockEtCommandes">

  <property name="geometry">

   <rect>

    <x>0</x>

    <y>0</y>

    <width>1000</width>

    <height>700</height>

   </rect>

  </property>

  <property name="windowTitle">

   <string>Gestion du Stock et des Commandes</string>

  </property>

  <layout class="QVBoxLayout" name="verticalLayout">
```

```
<item>
 <widget class="QTabWidget" name="tabWidget">
  <property name="currentIndex">
   <number>0</number>
  </property>


  <widget class="QWidget" name="tab_stock">
   <attribute name="title">
    <string>Gestion des Pièces en Stock</string>
   </attribute>
   <layout class="QVBoxLayout" name="verticalLayout_2">
    <item>
     <widget class="QGroupBox" name="groupBox_ajout_piece">
      <property name="title">
       <string>Ajouter une nouvelle Pièce (PIECE)</string>
      </property>
      <layout class="QGridLayout" name="gridLayout">
       <item row="0" column="0">
        <widget class="QLabel" name="label_nom_piece">
```

```
  <property name="text">
   <string>Nom de la Pièce :</string>
  </property>
 </widget>
</item>
<item row="0" column="1">
 <widget class="QLineEdit" name="input_nom_piece"/>
</item>
<item row="0" column="2">
 <widget class="QLabel" name="label_ref_piece">
  <property name="text">
   <string>Référence Fournisseur :</string>
  </property>
 </widget>
</item>
<item row="0" column="3">
 <widget class="QLineEdit" name="input_ref_piece"/>
</item>
<item row="1" column="0">
```

```
 <widget class="QLabel" name="label_prix">
  <property name="text">
   <string>Prix Unitaire HT :</string>
  </property>
 </widget>
</item>
<item row="1" column="1">
 <widget class="QLineEdit" name="input_prix_unitaire"/>
</item>
<item row="1" column="2">
 <widget class="QLabel" name="label_quantite">
  <property name="text">
   <string>Stock Initial :</string>
  </property>
 </widget>
</item>
<item row="1" column="3">
 <widget class="QLineEdit" name="input_quantite_stock"/>
</item>
```

```
   <item row="2" column="3">
    <widget class="QPushButton" name="btn_ajouter_piece">
     <property name="text">
      <string>Ajouter Pièce au Stock</string>
     </property>
    </widget>
   </item>
  </layout>
 </widget>
</item>
<item>
 <widget class="QLabel" name="label_liste_pieces">
  <property name="text">
   <string>Inventaire des Pièces :</string>
  </property>
 </widget>
</item>
<item>
 <widget class="QTableWidget" name="table_pieces">
```

```
<property name="columnCount">
 <number>5</number>
</property>
<column>
 <property name="text">
  <string>ID</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Nom</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Référence</string>
 </property>
</column>
<column>
```

```
     <property name="text">
      <string>Prix Unitaire</string>
     </property>
    </column>
    <column>
     <property name="text">
      <string>Quantité Stock</string>
     </property>
    </column>
   </widget>
  </item>
 </layout>
</widget>

<widget class="QWidget" name="tab_commandes">
 <attribute name="title">
  <string>Commandes Fournisseurs</string>
 </attribute>
 <layout class="QVBoxLayout" name="verticalLayout_3">
```

```
<item>
 <widget class="QGroupBox" name="groupBox_nouvelle_commande">
  <property name="title">
   <string>Nouvelle Commande de Pièces (COMMANDE_REPARATION)</string>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="label_fournisseur">
     <property name="text">
      <string>Fournisseur :</string>
     </property>
    </widget>
   </item>
   <item row="0" column="1">
    <widget class="QComboBox" name="combo_fournisseur_commande"/>
   </item>
   <item row="1" column="0">
    <widget class="QLabel" name="label_montant_estime">
     <property name="text">
```

```
      <string>Montant Estimé HT :</string>
     </property>
    </widget>
   </item>
   <item row="1" column="1">
    <widget class="QLineEdit" name="input_montant_commande"/>
   </item>
   <item row="2" column="1">
    <widget class="QPushButton" name="btn_passer_commande">
     <property name="text">
      <string>Passer la Commande (Ajouter Pièces)</string>
     </property>
    </widget>
   </item>
  </layout>
 </widget>
</item>
<item>
 <widget class="QLabel" name="label_commandes_en_cours">
```

```
  <property name="text">
   <string>Liste des Commandes :</string>
  </property>
 </widget>
</item>
<item>
 <widget class="QTableWidget" name="table_commandes">
  <property name="columnCount">
   <number>5</number>
  </property>
  <column>
   <property name="text">
    <string>ID Commande</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Fournisseur</string>
   </property>
```

```
  </column>
  <column>
   <property name="text">
    <string>Date</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Montant Est.</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Statut</string>
   </property>
  </column>
 </widget>
</item>
<item>
```

```
      <widget class="QPushButton" name="btn_recevoir_commande">
       <property name="text">
        <string>Recevoir Commande Sélectionnée et Mettre à Jour le Stock</string>
       </property>
      </widget>
     </item>
    </layout>
   </widget>
  </widget>
 </item>
</layout>
</widget>
<resources/>
<connections/>
</ui>
```

Vous avez fourni le code **XML** du fichier d'interface utilisateur **`GestionStockEtCommandes.ui`**, qui définit la structure de l'interface graphique pour la classe `GestionStockEtCommandes` en PyQt6.

Cette interface est conçue pour séparer les fonctionnalités de gestion du stock et des commandes grâce à un widget à onglets (`QTabWidget`).

Structure Principale de l'Interface

L'interface est encapsulée dans un **`QTabWidget`** nommé **`tabWidget`**, qui contient deux onglets principaux :

---

1. Onglet "Gestion des Pièces en Stock" (`tab_stock`)

Cet onglet gère l'inventaire des pièces de rechange (table `PIECE`).

Formulaire d'Ajout de Pièce (`groupBox_ajout_piece`)

Utilise un **`QGridLayout`** pour organiser la saisie des informations d'une nouvelle pièce :

Label	Widget (Nom)	Rôle dans `GestionStockEtCommandes.py`
Nom de la Pièce	**`QLineEdit`** (`input_nom_piece`)	Nom de la pièce.
Référence Fournisseur	**`QLineEdit`** (`input_ref_piece`)	Référence unique de la pièce.
Prix Unitaire HT	**`QLineEdit`** (`input_prix_unitaire`)	Prix de la pièce pour le calcul des coûts.
Stock Initial	**`QLineEdit`** (`input_quantite_stock`)	Quantité initiale en stock.
Bouton d'Action	**`QPushButton`** (`btn_ajouter_piece`)	Déclenche la méthode `ajouter_piece()`.

Tableau d'Inventaire

- **`QTableWidget`** nommé **`table_pieces`**.
- Affiche la liste complète des pièces en stock, chargée par `charger_pieces()`.
- Colonnes affichées : **ID, Nom, Référence, Prix Unitaire, Quantité Stock**.

---

2. Onglet "Commandes Fournisseurs" (`tab_commandes`)

Cet onglet gère le processus de commande et de réception des pièces (tables `COMMANDE_REPARATION` et `FOURNISSEUR`).

Formulaire de Nouvelle Commande (`groupBox_nouvelle_commande`)

Utilise un **`QFormLayout`** pour la saisie de base d'une nouvelle commande :

Label	Widget (Nom)	Rôle dans `GestionStockEtCommandes.py`
Fournisseur	**`QComboBox`** (`combo_fournisseur_commande`)	Liste déroulante chargée avec les fournisseurs disponibles par `charger_fournisseurs()`.
Montant Estimé HT	**`QLineEdit`** (`input_montant_commande`)	Montant total estimé de la commande.
Bouton d'Action	**`QPushButton`** (`btn_passer_commande`)	Déclenche la méthode `passer_commande_reparation()`.

Tableau des Commandes et Réception

- **`QTableWidget`** nommé **`table_commandes`**.
- Affiche l'historique et le statut des commandes fournisseurs, chargé par `charger_commandes()`.
- Colonnes affichées : **ID Commande, Fournisseur, Date, Montant Est., Statut**.
- **Bouton d'Action** : **`QPushButton`** (`btn_recevoir_commande`)
  - Permet de marquer une commande sélectionnée comme reçue.
  - Déclenche la méthode `recevoir_commande()`, qui met à jour le statut de la commande et incrémente le stock des pièces concernées.

code du fichier GestionVehicule.py : import sqlite3

```
from PyQt6.QtCore import pyqtSignal # Ajout de pyqtSignal

from PyQt6.QtWidgets import QWidget, QMessageBox, QTableWidgetItem, QTreeWidgetItem

from PyQt6.uic import loadUi



class GestionVehicule(QWidget):



    # SIGNAL Émis lorsque le statut d'un véhicule change manuellement

    vehicule_modifie = pyqtSignal()



    def __init__(self, db_connect_func):
```

```
    super().__init__()
    loadUi("GestionVehicule.ui", self)

    self.db_connect = db_connect_func

    # Connexion des boutons
    self.btn_ajouter.clicked.connect(self.ajouter_vehicule)
    self.btn_modifier.clicked.connect(self.modifier_vehicule)
    self.btn_supprimer.clicked.connect(self.supprimer_vehicule)
    self.tableVehicules.itemSelectionChanged.connect(self.charger_champs_vehicule)

    # Initialisation des données
    self.charger_types_vehicule()
    self.load_vehicules()

def charger_types_vehicule(self):
    """Charge les types de véhicules (TYPE_VEHICULE) dans le QComboBox."""
    self.combo_type_vehicule.clear()
    self.combo_type_vehicule.addItem("Sélectionner un type...")
```

```
try:
    conn = self.db_connect()
    cursor = conn.cursor()

    cursor.execute("SELECT id_type, nom_type FROM TYPE_VEHICULE ORDER BY nom_type")
    types_data = cursor.fetchall()

    for id_type, nom_type in types_data:
        self.combo_type_vehicule.addItem(nom_type, id_type)

    # Remplissage du combo_statut
    if self.combo_statut.count() == 0:
        self.combo_statut.addItems(['Disponible', 'En course', 'En réparation'])

except sqlite3.OperationalError as e:
    QMessageBox.critical(self, "Erreur DB", f"Impossible de charger les types de véhicules. La table TYPE_VEHICULE existe-t-elle? Détail: {e}")
except Exception as e:
    QMessageBox.critical(self, "Erreur Inconnue", f"Une erreur s'est produite lors du chargement: {e}")
```

```
    finally:
        if 'conn' in locals() and conn:
            conn.close()


def load_vehicules(self):
    """Charge tous les véhicules dans le QTreeWidget."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()
        cursor.execute("""
            SELECT V.immatriculation, V.marque, V.modele, V.kilometrage_actuel, T.nom_type, V.statut_disponibilite
            FROM VEHICULE V
            JOIN TYPE_VEHICULE T ON V.id_type = T.id_type
            ORDER BY V.immatriculation
        """)
        vehicules = cursor.fetchall()


        self.tableVehicules.clear()
```

```
        for row_data in vehicules:
            item = QTreeWidgetItem(self.tableVehicules)
            for col_num, data in enumerate(row_data):
                item.setText(col_num, str(data))

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des véhicules: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()

def charger_champs_vehicule(self):
    """Remplit les champs du formulaire avec les données du véhicule sélectionné."""
    selected_item = self.tableVehicules.currentItem()
    if selected_item:
        # Assumer les colonnes : 0=immat, 1=marque, 2=modele, 3=km, 4=type, 5=statut
        self.input_immat.setText(selected_item.text(0))
        self.input_marque.setText(selected_item.text(1))
        self.input_modele.setText(selected_item.text(2))
```

```
self.input_kilometrage.setText(selected_item.text(3))


# Positionner les ComboBox
self.combo_type_vehicule.setCurrentText(selected_item.text(4))


self.combo_statut.setCurrentText(selected_item.text(5))


# # self.combo_statut.setCurrentText(selected_item.text(5))
#
#
#
#
#
# #### DEBUT  ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :"   ::::::::
# #### DEBUT  ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :"   ::::::::
# #### DEBUT  ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :"   ::::::::
# #### DEBUT  ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :"   ::::::::
# #### DEBUT  ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :"   ::::::::
# # Dans GestionVehicule.py, dans la fonction charger_champs_vehicule:
```

```
        # # ...

        # # ...

        # # ...

        # # ...

        # statut = selected_item.text(5)

        # self.combo_statut.setCurrentText(statut)


        # # VERROUILLAGE : Si le véhicule est utilisé ou réparé, l'utilisateur ne peut pas forcer le changement.

        # # if statut in ('Réservé', 'En course', 'En réparation'):

        # # if statut in ('En course', 'En réparation'):

        # if statut in ('En course'):

        #     self.combo_statut.setEnabled(False)

        #     QMessageBox.warning(self, "Statut Verrouillé",

        #         # f"Le statut est géré par les modules 'Voyages' ou 'Réparations' car le véhicule est '{statut}'.")

        #         f"Le statut du Véhicule immatriculé {selected_item.text(0)} est géré par les modules 'Voyages' ou 'Réparations' car le véhicule est '{statut}'.")

        # else:

        #     self.combo_statut.setEnabled(True)

        # # ...
```

...

...

...

FIN ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :" !!!!!!!!

FIN ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :" !!!!!!!!

FIN ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :" !!!!!!!!

FIN ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :" !!!!!!!!

FIN ========> Dans l'Onglet ====> "Gestion des Véhicules" , dans la LISTE DEROULANTE ====> "Statut de Disponobilité :" !!!!!!!!

######## On décide de """"""DESACTIVER"""""" la """"""LISTE DEROULANTE"""""" nommée """"""combo_statut"""""" ::::::::::::

######## On décide de """"""DESACTIVER"""""" la """"""LISTE DEROULANTE"""""" nommée """"""combo_statut"""""" ::::::::::::

######## On décide de """"""DESACTIVER"""""" la """"""LISTE DEROULANTE"""""" nommée """"""combo_statut"""""" ::::::::::::

######## On décide de """"""DESACTIVER"""""" la """"""LISTE DEROULANTE"""""" nommée """"""combo_statut"""""" ::::::::::::

self.combo_statut.setEnabled(False)

```
    else:
        # Vider les champs
        self.input_immat.clear()
        self.input_marque.clear()
        self.input_modele.clear()
        self.input_kilometrage.clear()



def ajouter_vehicule(self):
    """Ajoute un nouveau véhicule à la flotte (VEHICULE)."""
    try:
        immat = self.input_immat.text().strip().upper()
        marque = self.input_marque.text().strip()
        modele = self.input_modele.text().strip()
        kilometrage = int(self.input_kilometrage.text().strip() or 0)
        type_vehicule_nom = self.combo_type_vehicule.currentText()
        statut = self.combo_statut.currentText()

        if not all([immat, marque, modele, type_vehicule_nom]) or type_vehicule_nom == "Sélectionner un type...":
```

```
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez remplir tous les champs obligatoires.")
    return

conn = self.db_connect()
cursor = conn.cursor()

# Récupérer l'ID du type
cursor.execute("SELECT id_type FROM TYPE_VEHICULE WHERE nom_type = ?", (type_vehicule_nom,))
id_type = cursor.fetchone()
if not id_type:
    QMessageBox.critical(self, "Erreur", "Type de véhicule non trouvé.")
    return

cursor.execute("""
    INSERT INTO VEHICULE (immatriculation, marque, modele, kilometrage_actuel, id_type, statut_disponibilite)
    VALUES (?, ?, ?, ?, ?, ?)
""", (immat, marque, modele, kilometrage, id_type[0], statut))

conn.commit()
```

```
        QMessageBox.information(self, "Succès", f"Le véhicule {immat} a été ajouté à la flotte.")
        self.load_vehicules()

        # 🎯 Émettre le signal pour mettre à jour l'onglet Voyage
        self.vehicule_modifie.emit()

    except ValueError:
        QMessageBox.critical(self, "Erreur de Format", "Le kilométrage doit être un nombre entier.")
    except sqlite3.IntegrityError:
        QMessageBox.critical(self, "Erreur SQL", f"L'immatriculation {immat} existe déjà.")
    finally:
        if 'conn' in locals() and conn:
            conn.close()


def modifier_vehicule(self):
    """Modifie les informations du véhicule sélectionné (VEHICULE)."""
    selected_item = self.tableVehicules.currentItem()
    if not selected_item:
```

```
    QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un véhicule dans le tableau à modifier.")
    return

old_immat = selected_item.text(0)

try:
    new_immat = self.input_immat.text().strip().upper()
    marque = self.input_marque.text().strip()
    modele = self.input_modele.text().strip()
    kilometrage = int(self.input_kilometrage.text().strip() or 0)
    type_vehicule_nom = self.combo_type_vehicule.currentText()
    statut = self.combo_statut.currentText()

    conn = self.db_connect()
    cursor = conn.cursor()

    cursor.execute("SELECT id_type FROM TYPE_VEHICULE WHERE nom_type = ?", (type_vehicule_nom,))
    id_type = cursor.fetchone()
```

```
    cursor.execute("""
        UPDATE VEHICULE SET
        immatriculation = ?, marque = ?, modele = ?, kilometrage_actuel = ?, id_type = ?, statut_disponibilite = ?
        WHERE immatriculation = ?
    """, (new_immat, marque, modele, kilometrage, id_type[0], statut, old_immat))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Le véhicule {old_immat} a été modifié.")
    self.load_vehicules()

    # 🎯 Émettre le signal pour mettre à jour l'onglet Voyage
    self.vehicule_modifie.emit()

except ValueError:
    QMessageBox.critical(self, "Erreur de Format", "Le kilométrage doit être un nombre entier.")
except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de modifier le véhicule: {e}")
finally:
    if 'conn' in locals() and conn:
```

```
        conn.close()



def supprimer_vehicule(self):
    """Supprime le véhicule sélectionné (VEHICULE)."""
    selected_item = self.tableVehicules.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un véhicule à supprimer.")
        return

    immat_a_supprimer = selected_item.text(0)

    reply = QMessageBox.question(self, 'Confirmation',
                                 f"Êtes-vous sûr de vouloir supprimer le véhicule {immat_a_supprimer}?",
                                 QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No, QMessageBox.StandardButton.No)

    if reply == QMessageBox.StandardButton.Yes:
        try:
            conn = self.db_connect()
```

```
    cursor = conn.cursor()

    cursor.execute("DELETE FROM VEHICULE WHERE immatriculation = ?", (immat_a_supprimer,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Le véhicule {immat_a_supprimer} a été supprimé.")
    self.load_vehicules()

    # 🎯 Émettre le signal pour mettre à jour l'onglet Voyage
    self.vehicule_modifie.emit()

except sqlite3.IntegrityError:
    QMessageBox.critical(self, "Erreur de Suppression", "Impossible de supprimer ce véhicule. Il est lié à des voyages ou des réparations.")
except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de supprimer le véhicule: {e}")
finally:
    if 'conn' in locals() and conn:
        conn.close()
```

Le code fourni est le fichier Python **`GestionVehicule.py`**, qui implémente la classe `GestionVehicule` pour gérer l'ajout, la modification, la suppression et l'affichage des véhicules de la flotte.

Cette classe interagit principalement avec les tables de la base de données **`VEHICULE`** et **`TYPE_VEHICULE`**.

🚀 Fonctionnalités Principales

1. Signalisation et Initialisation

- **`vehicule_modifie = pyqtSignal()`** : Déclare un **signal** personnalisé qui est émis chaque fois qu'un véhicule est ajouté, modifié ou supprimé. Ce signal est crucial pour informer les autres modules (comme la gestion des voyages) qu'ils doivent rafraîchir leurs listes de véhicules.
- **`__init__`**: Charge l'interface **`GestionVehicule.ui`**, connecte les trois boutons d'action (`btn_ajouter`, `btn_modifier`, `btn_supprimer`) aux méthodes correspondantes, et connecte la sélection dans le tableau (`tableVehicules`) à la méthode `charger_champs_vehicule()`.

---

2. Chargement des Données

- **`charger_types_vehicule()`** :
  - Récupère les `id_type` et `nom_type` de la table **`TYPE_VEHICULE`**.
  - Remplit la liste déroulante **`combo_type_vehicule`**.
  - **Initialise également** **`combo_statut`** avec les options 'Disponible', 'En course', et 'En réparation'.
- **`load_vehicules()`** :
  - Effectue une jointure entre **`VEHICULE`** et **`TYPE_VEHICULE`** pour obtenir toutes les informations complètes du véhicule.
  - Affiche les données (`immatriculation`, `marque`, `modele`, `kilometrage_actuel`, `nom_type`, `statut_disponibilite`) dans le **`QTreeWidget`** nommé **`tableVehicules`**.
- **`charger_champs_vehicule()`** :
  - Déclenché lorsque l'utilisateur clique sur une ligne du tableau `tableVehicules`.
  - Remplit automatiquement les champs de saisie (`input_immat`, `input_marque`, `input_modele`, `input_kilometrage`) et positionne les listes déroulantes (`combo_type_vehicule`, `combo_statut`) avec les données du véhicule sélectionné.
  - **Note Importante** : La liste déroulante **`combo_statut` est désactivée (`setEnabled(False)`)** manuellement après le chargement des données, indiquant que le statut de disponibilité ne doit pas être modifié directement par l'utilisateur dans ce module, mais géré par les modules `Voyages` ou `Réparations`.

---

3. Gestion des CRUD (Création, Lecture, Mise à jour, Suppression)

- **`ajouter_vehicule()`** :
  - Récupère les données du formulaire, y compris l'ID du type de véhicule.
  - **Insère** une nouvelle ligne dans la table **`VEHICULE`**.
  - Émet le signal **`vehicule_modifie`** pour notifier les autres modules.
  - Gère les erreurs de format (kilométrage) et d'intégrité (immatriculation déjà existante).
- **`modifier_vehicule()`** :
  - Vérifie qu'un véhicule est sélectionné.
  - **Met à jour** les champs de la ligne correspondante dans la table **`VEHICULE`** en utilisant l'ancienne immatriculation (`old_immat`) comme clé de recherche.
  - Émet le signal **`vehicule_modifie`**.
- **`supprimer_vehicule()`** :
  - Demande confirmation à l'utilisateur.
  - **Supprime** la ligne de la table **`VEHICULE`** correspondant à l'immatriculation sélectionnée.
  - Gère l'erreur d'intégrité si le véhicule est lié à d'autres enregistrements (voyages ou réparations).
  - Émet le signal **`vehicule_modifie`**.

code du fichier GestionVehicule.ui : <?xml version="1.0" encoding="UTF-8"?>


```
<ui version="4.0">

 <class>GestionVehicule</class>

 <widget class="QWidget" name="GestionVehicule">

  <property name="geometry">

   <rect>

    <x>0</x>

    <y>0</y>

    <width>750</width>

    <height>600</height>
```

```
 </rect>
</property>
<property name="windowTitle">
 <string>Gestion de la Flotte</string>
</property>
<layout class="QVBoxLayout" name="mainLayout">
 <item>
  <widget class="QLabel" name="label_titre">
   <property name="styleSheet">
    <string notr="true">font: 18pt "Times New Roman"; color: rgb(0, 85, 127);</string>
   </property>
   <property name="text">
    <string>Gestion des Véhicules</string>
   </property>
   <property name="alignment">
    <set>Qt::AlignCenter</set>
   </property>
  </widget>
 </item>
```

```
<item>
 <widget class="QGroupBox" name="formGroup">
  <property name="title">
   <string>Détails du Véhicule</string>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="label_immat">
     <property name="text">
      <string>&amp;amp;Immatriculation :</string>
     </property>
     <property name="buddy">
      <cstring>input_immat</cstring>
     </property>
    </widget>
   </item>
   <item row="0" column="1">
    <widget class="QLineEdit" name="input_immat"/>
   </item>
```

```
<item row="1" column="0">
 <widget class="QLabel" name="label_marque">
  <property name="text">
   <string>&amp;amp;Marque :</string>
  </property>
  <property name="buddy">
   <cstring>input_marque</cstring>
  </property>
 </widget>
</item>
<item row="1" column="1">
 <widget class="QLineEdit" name="input_marque"/>
</item>
<item row="2" column="0">
 <widget class="QLabel" name="label_modele">
  <property name="text">
   <string>Mo&amp;amp;dèle :</string>
  </property>
  <property name="buddy">
```

```
   <cstring>input_modele</cstring>
  </property>
 </widget>
</item>
<item row="2" column="1">
 <widget class="QLineEdit" name="input_modele"/>
</item>
<item row="3" column="0">
 <widget class="QLabel" name="label_kilometrage">
  <property name="text">
   <string>&amp;amp;Kilométrage Actuel :</string>
  </property>
  <property name="buddy">
   <cstring>input_kilometrage</cstring>
  </property>
 </widget>
</item>
<item row="3" column="1">
 <widget class="QLineEdit" name="input_kilometrage"/>
```

```
</item>
<item row="4" column="0">
 <widget class="QLabel" name="label_type">
  <property name="text">
   <string>&amp;amp;Type de Véhicule :</string>
  </property>
  <property name="buddy">
   <cstring>combo_type_vehicule</cstring>
  </property>
 </widget>
</item>
<item row="4" column="1">
 <widget class="QComboBox" name="combo_type_vehicule"/>
</item>
<item row="5" column="0">
 <widget class="QLabel" name="label_statut">
  <property name="text">
   <string>&amp;amp;Statut de Disponibilité :</string>
  </property>
```

```
    <property name="buddy">
     <cstring>combo_statut</cstring>
    </property>
   </widget>
  </item>
  <item row="5" column="1">
   <widget class="QComboBox" name="combo_statut"/>
  </item>
 </layout>
</widget>
</item>
<item>
 <layout class="QHBoxLayout" name="buttonLayout">
  <item>
   <spacer name="horizontalSpacer_left">
    <property name="orientation">
     <enum>Qt::Horizontal</enum>
    </property>
    <property name="sizeHint" stdset="0">
```

```
   <size>
    <width>40</width>
    <height>20</height>
   </size>
  </property>
 </spacer>
</item>
<item>
 <widget class="QPushButton" name="btn_ajouter">
  <property name="text">
   <string>Ajouter</string>
  </property>
 </widget>
</item>
<item>
 <widget class="QPushButton" name="btn_modifier">
  <property name="text">
   <string>Modifier</string>
  </property>
```

```
 </widget>
</item>
<item>
 <widget class="QPushButton" name="btn_supprimer">
  <property name="text">
   <string>Supprimer</string>
  </property>
 </widget>
</item>
<item>
 <spacer name="horizontalSpacer_right">
  <property name="orientation">
   <enum>Qt::Horizontal</enum>
  </property>
  <property name="sizeHint" stdset="0">
   <size>
    <width>40</width>
    <height>20</height>
   </size>
```

```
    </property>
   </spacer>
  </item>
 </layout>
</item>
<item>
 <widget class="QTreeWidget" name="tableVehicules">
  <column>
   <property name="text">
    <string>Immatriculation</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Marque</string>
   </property>
  </column>
  <column>
   <property name="text">
```

```
  <string>Modèle</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Kilométrage</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Type</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Statut</string>
 </property>
</column>
</widget>
```

```
  </item>

 </layout>

</widget>

<resources/>

<connections/>

</ui>
```

Vous avez fourni le code **XML** du fichier d'interface utilisateur **`GestionVehicule.ui`**. Ce fichier définit la mise en page et les composants graphiques utilisés par la classe Python `GestionVehicule` pour gérer la flotte.

Structure de l'Interface (**`GestionVehicule.ui`**)

L'interface utilise un **`QVBoxLayout`** (mise en page verticale principale) pour organiser trois sections principales : le titre, le formulaire de détails, les boutons d'action et le tableau d'affichage.

1. Formulaire de Détails du Véhicule (**`formGroup`**)

Ce formulaire permet la saisie et la modification des informations d'un véhicule. Il est organisé à l'aide d'un **`QFormLayout`** (étiquette à gauche, champ de saisie à droite).

Label (Étiquette)	Widget (Type / Nom)	Description
Immatriculation	**`QLineEdit`** (`input_immat`)	Champ de texte pour le numéro d'immatriculation (clé primaire).
Marque	**`QLineEdit`** (`input_marque`)	Champ de texte pour la marque du véhicule.
Modèle	**`QLineEdit`** (`input_modele`)	Champ de texte pour le modèle.
Kilométrage Actuel	**`QLineEdit`** (`input_kilometrage`)	Champ de texte pour le kilométrage (saisi comme un entier dans le code Python).
Type de Véhicule	**`QComboBox`** (`combo_type_vehicule`)	Liste déroulante alimentée par la table `TYPE_VEHICULE`.

Label (Étiquette)	Widget (Type / Nom)	Description
Statut de Disponibilité	**`QComboBox`** (`combo_statut`)	Liste déroulante contenant les statuts (`Disponible`, `En course`, `En réparation`). Notez que le code Python **désactive** ce champ après le chargement pour éviter les modifications manuelles non intentionnelles du statut.

---

2. Boutons d'Action (`buttonLayout`)

Ces boutons déclenchent les fonctions de gestion des données (CRUD) dans le fichier `GestionVehicule.py`.

- **`QPushButton`** (`btn_ajouter`) : Connecté à `ajouter_vehicule()`.
- **`QPushButton`** (`btn_modifier`) : Connecté à `modifier_vehicule()`.
- **`QPushButton`** (`btn_supprimer`) : Connecté à `supprimer_vehicule()`.

---

3. Tableau des Véhicules

- **`QTreeWidget`** nommé **`tableVehicules`**.
- Ce widget affiche la liste complète des véhicules chargés par `load_vehicules()`.
- La sélection d'un élément dans ce tableau déclenche la fonction `charger_champs_vehicule()` qui remplit automatiquement le formulaire.
- Les colonnes définies correspondent aux données affichées : **Immatriculation, Marque, Modèle, Kilométrage, Type, Statut**.

code du fichier GestionVoyage.py : import sqlite3

from PyQt6.QtCore import pyqtSignal

from PyQt6.QtWidgets import QWidget, QMessageBox, QTreeWidgetItem

from PyQt6.uic import loadUi

from datetime import datetime

class GestionVoyage(QWidget):

```
# SIGNAL : Émis lorsque le statut d'un véhicule change suite à la fin d'un voyage.
voyage_termine = pyqtSignal()

def __init__(self, db_connect_func):
    super().__init__()
    # Chargement de l'interface utilisateur
    loadUi("GestionVoyage.ui", self)

    self.db_connect = db_connect_func

    # Connexion des boutons
    self.btn_planifier.clicked.connect(self.planifier_voyage)
    self.btn_terminer.clicked.connect(self.terminer_voyage)
    self.btn_facturer.clicked.connect(self.generer_facture)

    # Connexion de l'événement de sélection pour remplir les champs
    self.tableVoyages.itemSelectionChanged.connect(self.charger_champs_voyage)
```

```
    # Initialisation des données
    self.charger_comboboxes()
    self.load_voyages()

def charger_comboboxes(self):
    """[SLOT] Charge les listes déroulantes (clients, chauffeurs, véhicules disponibles)."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()

        # --- Clients (avec ID stocké) ---
        cursor.execute("SELECT id_client, nom || ' ' || prenom FROM CLIENT ORDER BY nom, prenom")
        clients = cursor.fetchall()
        self.combo_client.clear()
        self.combo_client.addItem("Sélectionner un client...", -1)

        for id_client, nom_complet in clients:
            self.combo_client.addItem(nom_complet, id_client)
```

```
    # --- Chauffeurs (avec ID stocké) ---
    cursor.execute("SELECT id_chauffeur, nom || ' ' || prenom FROM CHAUFFEUR ORDER BY nom, prenom")
    chauffeurs = cursor.fetchall()
    self.combo_chauffeur.clear()
    self.combo_chauffeur.addItem("Sélectionner un chauffeur...", -1)

    for id_chauffeur, nom_complet in chauffeurs:
        self.combo_chauffeur.addItem(nom_complet, id_chauffeur)

    # --- Véhicules (uniquement Disponibles) ---
    cursor.execute("SELECT immatriculation FROM VEHICULE WHERE statut_disponibilite = 'Disponible'")
    vehicules = [row[0] for row in cursor.fetchall()]
    self.combo_vehicule.clear()
    self.combo_vehicule.addItem("Sélectionner un véhicule disponible...")
    self.combo_vehicule.addItems(vehicules)

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des ComboBoxes: {e}")
finally:
```

```
        if 'conn' in locals() and conn:
            conn.close()


# def load_voyages(self):
#     """[SLOT] Charge tous les voyages dans le QTreeWidget."""
#     try:
#         conn = self.db_connect()
#         cursor = conn.cursor()
#         # Note : distance_estimee n'est pas affichée ici, mais peut être ajoutée au SELECT/setHeaderLabels si nécessaire.
#         cursor.execute("""
#             SELECT
#                 V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
#                 V.lieu_depart, V.lieu_arrivee, V.date_fin_reelle
#             FROM VOYAGE V
#             JOIN CLIENT C ON V.id_client = C.id_client
#             JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
#             JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
#             ORDER BY V.id_voyage DESC
#         """)
```

```
#        voyages = cursor.fetchall()

#        self.tableVoyages.clear()

#        self.tableVoyages.setHeaderLabels([
#            "ID Voyage", "Client", "Chauffeur", "Immatriculation",
#            "Statut", "Départ", "Arrivée", "Date Fin"
#        ])

#        for row_data in voyages:
#            item = QTreeWidgetItem(self.tableVoyages)
#            for col_num, data in enumerate(row_data):
#                item.setText(col_num, str(data if data is not None else ''))

#    except sqlite3.Error as e:
#        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
#    finally:
#        if 'conn' in locals() and conn:
#            conn.close()
```

```
# def load_voyages(self):
#         """[SLOT] Charge tous les voyages dans le QTreeWidget."""
#         try:
#             conn = self.db_connect()
#             cursor = conn.cursor()

#             # --- MODIFICATION ICI : Ajout de V.date_debut_planifiee ---
#             cursor.execute("""
#                 SELECT
#                     V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
#                     V.lieu_depart, V.lieu_arrivee, V.date_debut_planifiee, V.date_fin_reelle
#                 FROM VOYAGE V
#                 JOIN CLIENT C ON V.id_client = C.id_client
#                 JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
#                 JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
#                 ORDER BY V.id_voyage DESC
#             """)
#             voyages = cursor.fetchall()
```

```
#           self.tableVoyages.clear()

#           # --- MODIFICATION CI-DESSOUS : Ajustement du header ---
#           self.tableVoyages.setHeaderLabels([
#               "ID Voyage", "Client", "Chauffeur", "Immatriculation",
#               "Statut", "Départ", "Arrivée", "Date Début", "Date Fin" # Nouvelle colonne "Date Début"
#           ])

#           for row_data in voyages:
#               item = QTreeWidgetItem(self.tableVoyages)
#               for col_num, data in enumerate(row_data):
#                   item.setText(col_num, str(data if data is not None else ''))

#       except sqlite3.Error as e:
#           QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
#       finally:
#           if 'conn' in locals() and conn:
#               conn.close()
```

```
def load_voyages(self):
    """[SLOT] Charge tous les voyages dans le QTreeWidget."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()

        # ATTENTION : La requête originale n'a que 8 colonnes.
        # Nous devons l'ajuster pour inclure la distance estimée et la date de début (si elle est nécessaire, basé sur l'UI).
        # Nous utiliserons 9 colonnes dans cet exemple pour inclure toutes les données pertinentes.

        cursor.execute("""
            SELECT
                V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
                V.lieu_depart, V.lieu_arrivee, V.date_debut_planifiee, V.distance_estimee, V.date_fin_reelle
            FROM VOYAGE V
            JOIN CLIENT C ON V.id_client = C.id_client
            JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
            JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
```

```
    ORDER BY V.id_voyage DESC
""")
voyages = cursor.fetchall()

self.tableVoyages.clear()

# --- ÉTAPE 2 : Mise à jour des en-têtes de colonnes (10 colonnes au total) ---
# L'ordre doit correspondre à la requête SQL.
self.tableVoyages.setHeaderLabels([
    "ID Voyage", "Client", "Chauffeur", "Immatriculation", "Statut",
    "Départ", "Arrivée", "Date Début", "Distance Estimée (km)", "Date Fin Réelle"
])

# --- ÉTAPE 3 : Remplissage du QTreeWidget ---
for row_data in voyages:
    item = QTreeWidgetItem(self.tableVoyages)
    for col_num, data in enumerate(row_data):
        item.setText(col_num, str(data if data is not None else ''))
```

```
        except sqlite3.Error as e:
            QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
        finally:
            if 'conn' in locals() and conn:
                conn.close()


def charger_champs_voyage(self):
    """Remplit les champs du formulaire avec les données du voyage sélectionné."""
    selected_item = self.tableVoyages.currentItem()
    if selected_item:
        try:
            # La colonne distance_estimee n'est pas directement affichée dans le QTreeWidget actuel
            # On ne peut donc pas la recharger directement sans modifier la requête load_voyages


            self.combo_client.setCurrentText(selected_item.text(1))
            self.combo_chauffeur.setCurrentText(selected_item.text(2))
        except:
            pass
```

```
def planifier_voyage(self):
    """Enregistre un nouveau voyage, inclut la distance estimée et met le véhicule 'En course'."""

    # Récupération des IDs
    id_client = self.combo_client.currentData()
    id_chauffeur = self.combo_chauffeur.currentData()

    immatriculation = self.combo_vehicule.currentText()
    depart = self.input_depart.text().strip()
    arrivee = self.input_arrivee.text().strip()
    date_debut = self.date_debut.dateTime().toString("yyyy-MM-dd HH:mm:ss")

    # 🎯 AJOUT / CORRECTION : Lecture et validation de la distance estimée
    distance_input = self.input_distance_estimee.text().strip()

    try:
        # Si le champ est vide, distance_estimee sera 0.0. Sinon, on tente de convertir.
        distance_estimee = float(distance_input) if distance_input else 0.0
```

```
    if distance_estimee < 0:
        QMessageBox.warning(self, "Erreur de Saisie", "La distance estimée ne peut pas être négative.")
        return
except ValueError:
    QMessageBox.warning(self, "Erreur de Saisie", "La distance estimée doit être un nombre valide.")
    return

if id_client is None or id_client == -1:
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez sélectionner un Client valide.")
    return

if id_chauffeur is None or id_chauffeur == -1:
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez sélectionner un Chauffeur valide.")
    return

if not all([immatriculation, depart, arrivee]) or immatriculation == "Sélectionner un véhicule disponible...":
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez remplir l'immatriculation, le lieu de départ et d'arrivée.")
    return
```

```
try:
    conn = self.db_connect()
    cursor = conn.cursor()

    # 🎯 CORRECTION : Mise à jour de la requête INSERT pour inclure distance_estimee
    cursor.execute("""
        INSERT INTO VOYAGE (id_client, immatriculation, id_chauffeur, statut_voyage,
                            lieu_depart, lieu_arrivee, date_debut_planifiee, distance_estimee)
        VALUES (?, ?, ?, 'Planifié', ?, ?, ?, ?)
    """, (id_client, immatriculation, id_chauffeur, depart, arrivee, date_debut, distance_estimee))

    # Mettre le statut du VEHICULE à 'En course'
    cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'En course' WHERE immatriculation = ?", (immatriculation,))

    conn.commit()
    QMessageBox.information(self, "Succès", "Voyage planifié avec succès. Statut du véhicule mis à jour.")
    self.load_voyages()
    self.charger_comboboxes()
```

```
        # Nettoyage des champs après succès
        self.input_depart.clear()
        self.input_arrivee.clear()
        self.input_distance_estimee.clear() # Nettoyage du nouveau champ
        self.combo_client.setCurrentIndex(0)
        self.combo_chauffeur.setCurrentIndex(0)
        self.combo_vehicule.setCurrentIndex(0)

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de planifier le voyage: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()

def terminer_voyage(self):
    """Clôture un voyage sélectionné, met à jour les données et remet le véhicule 'Disponible'."""
    selected_item = self.tableVoyages.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un voyage 'Planifié' à terminer.")
```

```
    return

id_voyage = selected_item.text(0)
statut_actuel = selected_item.text(4)
immatriculation = selected_item.text(3)

if statut_actuel != 'Planifié':
    QMessageBox.warning(self, "Statut Invalide", "Seuls les voyages 'Planifiés' peuvent être terminés.")
    return

try:
    conn = self.db_connect()
    cursor = conn.cursor()

    date_fin = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    # Mettre à jour le voyage
    cursor.execute("""
        UPDATE VOYAGE SET
```

```
        statut_voyage = 'Terminé',
        date_fin_reelle = ?
        WHERE id_voyage = ?
    """, (date_fin, id_voyage))

    # Mettre à jour le statut du VEHICULE à 'Disponible'
    cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'Disponible' WHERE immatriculation = ?", (immatriculation,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Voyage ID {id_voyage} terminé. Le véhicule est de nouveau disponible.")
    self.load_voyages()
    self.charger_comboboxes()

    # Émission du signal 'voyage_termine' pour synchroniser les autres modules
    self.voyage_termine.emit()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de terminer le voyage: {e}")
finally:
```

```
        if 'conn' in locals() and conn:
            conn.close()

def generer_facture(self):
    """Génère une facture pour un voyage terminé (FACTURE_CLIENT)."""
    selected_item = self.tableVoyages.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un voyage 'Terminé' à facturer.")
        return

    id_voyage = selected_item.text(0)
    statut_actuel = selected_item.text(4)

    if statut_actuel != 'Terminé':
        QMessageBox.warning(self, "Statut Invalide", "Seuls les voyages 'Terminés' peuvent être facturés.")
        return

    try:
        conn = self.db_connect()
```

```
    cursor = conn.cursor()

    # Vérifier si une facture existe déjà
    cursor.execute("SELECT id_facture FROM FACTURE_CLIENT WHERE id_voyage = ?", (id_voyage,))
    if cursor.fetchone():
        QMessageBox.information(self, "Information", f"Le voyage ID {id_voyage} a déjà été facturé.")
        return

    # Insérer la nouvelle facture (simplifié : insère juste l'ID voyage)
    cursor.execute("INSERT INTO FACTURE_CLIENT (id_voyage) VALUES (?)", (id_voyage,))

    # Mettre à jour le statut du voyage à 'Facturé'
    cursor.execute("UPDATE VOYAGE SET statut_voyage = 'Facturé' WHERE id_voyage = ?", (id_voyage,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Facture générée (ID Voyage: {id_voyage}). Statut mis à jour à 'Facturé'.")
    self.load_voyages()

except sqlite3.Error as e:
```

```
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de générer la facture: {e}")

    finally:

        if 'conn' in locals() and conn:

            conn.close()
```

Le code fourni est le fichier Python **`GestionVoyage.py`**, qui implémente la classe `GestionVoyage`. Cette classe gère le cycle de vie complet d'un voyage : **planification**, **terminaison** et **facturation**.

Elle interagit avec plusieurs tables de la base de données SQLite : **`VOYAGE`**, **`CLIENT`**, **`CHAUFFEUR`**, **`VEHICULE`** et **`FACTURE_CLIENT`**.

---

1. Initialisation et Chargement des Données

- **`voyage_termine = pyqtSignal()`** : Ce signal est émis lorsque la méthode `terminer_voyage()` est exécutée. Il sert à notifier d'autres modules (notamment `GestionVehicule`) que le statut d'un véhicule a changé (passé de 'En course' à 'Disponible').
- **`__init__`** : Charge l'interface **`GestionVoyage.ui`** et connecte les boutons aux fonctions :
  - `btn_planifier` → `planifier_voyage()`
  - `btn_terminer` → `terminer_voyage()`
  - `btn_facturer` → `generer_facture()`
  - Le changement de sélection dans `tableVoyages` → `charger_champs_voyage()`.
- **`charger_comboboxes()`** :
  - Charge les listes déroulantes pour les **clients** (`CLIENT`), les **chauffeurs** (`CHAUFFEUR`) en stockant leur `id_client` / `id_chauffeur` dans l'attribut `currentData`.
  - Charge la liste déroulante des **véhicules** (`VEHICULE`) en filtrant uniquement ceux ayant le `statut_disponibilite = 'Disponible'`.
- **`load_voyages()`** :
  - Récupère la liste de **tous les voyages** en utilisant une requête `JOIN` pour afficher les noms du client et du chauffeur, ainsi que l'immatriculation du véhicule.
  - Affiche les résultats dans le `QTreeWidget` (`tableVoyages`) avec les 10 colonnes suivantes : **ID Voyage, Client, Chauffeur, Immatriculation, Statut, Départ, Arrivée, Date Début, Distance Estimée (km), Date Fin Réelle**.

---

2. Planification d'un Voyage (`planifier_voyage`)

1. **Validation des Données** : Vérifie que le client, le chauffeur et le véhicule ont été sélectionnés, et que la distance estimée est un nombre valide (>= 0).
2. **Insertion `VOYAGE`** : Insère un nouvel enregistrement dans la table `VOYAGE` avec le `statut_voyage` initial **'Planifié'** et stocke la `distance_estimee`.
3. **Mise à Jour `VEHICULE`** : Met à jour le `statut_disponibilite` du véhicule sélectionné à **'En course'**.
4. **Rafraîchissement** : Recharge la liste des voyages et la liste des véhicules disponibles (via `charger_comboboxes()`) pour retirer le véhicule planifié de la liste des choix.

---

3. Clôture d'un Voyage (`terminer_voyage`)

1. **Vérification du Statut** : S'assure que le voyage sélectionné est bien **'Planifié'**.
2. **Mise à Jour `VOYAGE`** : Met à jour l'enregistrement du voyage :
   - `statut_voyage` passe à **'Terminé'**.
   - `date_fin_reelle` est enregistré à l'heure actuelle.
3. **Mise à Jour `VEHICULE`** : Remet le `statut_disponibilite` du véhicule concerné à **'Disponible'**.
4. **Signal** : Émet le signal **`voyage_termine`**.

---

4. Facturation (`generer_facture`)

1. **Vérification du Statut** : S'assure que le voyage sélectionné est **'Terminé'**.
2. **Vérification d'Existence** : Vérifie si une facture existe déjà dans la table `FACTURE_CLIENT` pour cet `id_voyage`.
3. **Insertion `FACTURE_CLIENT`** : Crée un nouvel enregistrement dans la table `FACTURE_CLIENT`.
4. **Mise à Jour `VOYAGE`** : Met à jour le `statut_voyage` du voyage à **'Facturé'**.

code du fichier GestionVoyage.ui : <?xml version="1.0" encoding="UTF-8"?>

<ui version="4.0">

 <class>GestionVoyage</class>

 <widget class="QWidget" name="GestionVoyage">

```
<property name="geometry">
 <rect>
  <x>0</x>
  <y>0</y>
  <width>750</width>
  <height>600</height>
 </rect>
</property>
<property name="windowTitle">
 <string>Gestion des Voyages</string>
</property>
<layout class="QVBoxLayout" name="mainLayout">
 <item>
  <widget class="QLabel" name="label_titre">
   <property name="styleSheet">
    <string notr="true">font: 18pt "Times New Roman"; color: rgb(0, 85, 127);</string>
   </property>
   <property name="text">
    <string>Planification et Réservation des Voyages</string>
```

```
  </property>
  <property name="alignment">
   <set>Qt::AlignCenter</set>
  </property>
 </widget>
</item>
<item>
 <widget class="QGroupBox" name="formGroup">
  <property name="title">
   <string>Détails du Voyage</string>
  </property>
  <layout class="QFormLayout" name="formLayout">
   <item row="0" column="0">
    <widget class="QLabel" name="label_client">
     <property name="text">
      <string>&amp;Client (ID/Nom) :</string>
     </property>
     <property name="buddy">
      <cstring>combo_client</cstring>
```

```
  </property>
 </widget>
</item>
<item row="0" column="1">
 <widget class="QComboBox" name="combo_client"/>
</item>
<item row="1" column="0">
 <widget class="QLabel" name="label_depart">
  <property name="text">
   <string>Lieu de &amp;Départ :</string>
  </property>
  <property name="buddy">
   <cstring>input_depart</cstring>
  </property>
 </widget>
</item>
<item row="1" column="1">
 <widget class="QLineEdit" name="input_depart"/>
</item>
```

```
<item row="2" column="0">
 <widget class="QLabel" name="label_arrivee">
  <property name="text">
   <string>Lieu d'&amp;amp;Arrivée :</string>
  </property>
  <property name="buddy">
   <cstring>input_arrivee</cstring>
  </property>
 </widget>
</item>
<item row="2" column="1">
 <widget class="QLineEdit" name="input_arrivee"/>
</item>
<item row="3" column="0">
 <widget class="QLabel" name="label_date_debut">
  <property name="text">
   <string>Date &amp;amp;Début :</string>
  </property>
  <property name="buddy">
```

```
   <cstring>date_debut</cstring>
  </property>
 </widget>
</item>
<item row="3" column="1">
 <widget class="QDateTimeEdit" name="date_debut"/>
</item>


<item row="4" column="0">
 <widget class="QLabel" name="label_distance">
  <property name="text">
   <string>Distance &amp;amp;Estimée (km) :</string>
  </property>
  <property name="buddy">
   <cstring>input_distance_estimee</cstring>
  </property>
 </widget>
</item>
<item row="4" column="1">
```

```
 <widget class="QLineEdit" name="input_distance_estimee"/>
</item>
<item row="5" column="0">
 <widget class="QLabel" name="label_chauffeur">
  <property name="text">
   <string>Chau&amp;amp;ffeur Attribué :</string>
  </property>
  <property name="buddy">
   <cstring>combo_chauffeur</cstring>
  </property>
 </widget>
</item>
<item row="5" column="1">
 <widget class="QComboBox" name="combo_chauffeur"/>
</item>
<item row="6" column="0">
 <widget class="QLabel" name="label_vehicule">
  <property name="text">
   <string>Véhicu&amp;amp;le Attribué :</string>
```

```
    </property>
    <property name="buddy">
     <cstring>combo_vehicule</cstring>
    </property>
   </widget>
  </item>
  <item row="6" column="1">
   <widget class="QComboBox" name="combo_vehicule"/>
  </item>
 </layout>
</widget>
</item>
<item>
 <layout class="QHBoxLayout" name="buttonLayout">
  <item>
   <spacer name="horizontalSpacer_left">
    <property name="orientation">
     <enum>Qt::Horizontal</enum>
    </property>
```

```
  <property name="sizeHint" stdset="0">
   <size>
    <width>40</width>
    <height>20</height>
   </size>
  </property>
 </spacer>
</item>
<item>
 <widget class="QPushButton" name="btn_planifier">
  <property name="text">
   <string>Planifier le Voyage</string>
  </property>
 </widget>
</item>
<item>
 <widget class="QPushButton" name="btn_terminer">
  <property name="text">
   <string>Terminer (Clôturer)</string>
```

```
  </property>
 </widget>
</item>
<item>
 <widget class="QPushButton" name="btn_facturer">
  <property name="text">
   <string>Générer Facture</string>
  </property>
 </widget>
</item>
<item>
 <spacer name="horizontalSpacer_right">
  <property name="orientation">
   <enum>Qt::Horizontal</enum>
  </property>
  <property name="sizeHint" stdset="0">
   <size>
    <width>40</width>
    <height>20</height>
```

```
    </size>
   </property>
  </spacer>
 </item>
</layout>
</item>
<item>
 <widget class="QTreeWidget" name="tableVoyages">
  <column>
   <property name="text">
    <string>ID</string>
   </property>
  </column>
  <column>
   <property name="text">
    <string>Client</string>
   </property>
  </column>
  <column>
```

```
 <property name="text">
  <string>Départ - Arrivée</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Date Début</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Distance Estimée (km)</string>
 </property>
</column>
<column>
 <property name="text">
  <string>Véhicule</string>
 </property>
</column>
```

```
    <column>
     <property name="text">
      <string>Statut</string>
     </property>
    </column>
   </widget>
  </item>
 </layout>
</widget>
<customwidgets/>
<resources/>
<connections/>
</ui>
```

Vous avez fourni le code **XML** du fichier d'interface utilisateur `GestionVoyage.ui`. Ce fichier définit l'agencement et les composants graphiques pour l'onglet de gestion des voyages.

Structure de l'Interface (`GestionVoyage.ui`)

L'interface est structurée verticalement à l'aide d'un `QVBoxLayout` et se divise en trois zones principales : le formulaire de planification, les boutons d'action et le tableau des voyages.

---

1. Formulaire de Planification (`formGroup`)

Le formulaire utilise un **`QFormLayout`** pour associer les étiquettes aux champs de saisie, permettant de planifier un nouveau voyage.

Label (Étiquette)	Widget (Type / Nom)	Description
Client (ID/Nom)	**`QComboBox`** (`combo_client`)	Liste déroulante pour sélectionner le client.
Lieu de Départ	**`QLineEdit`** (`input_depart`)	Champ de texte pour le lieu de départ.
Lieu d'Arrivée	**`QLineEdit`** (`input_arrivee`)	Champ de texte pour le lieu d'arrivée.
Date Début	**`QDateTimeEdit`** (`date_debut`)	Sélecteur de date et d'heure pour la planification.
Distance Estimée (km)	**`QLineEdit`** (`input_distance_estimee`)	**Nouveau champ** pour saisir la distance prévue.
Chauffeur Attribué	**`QComboBox`** (`combo_chauffeur`)	Liste déroulante pour sélectionner le chauffeur.
Véhicule Attribué	**`QComboBox`** (`combo_vehicule`)	Liste déroulante pour choisir le véhicule **disponible**.

---

2. Boutons d'Action (`buttonLayout`)

Ces boutons permettent de gérer le statut des voyages. Ils sont connectés aux méthodes Python correspondantes.

- **`QPushButton`** (`btn_planifier`) : Connecté à `planifier_voyage()`.
- **`QPushButton`** (`btn_terminer`) : Connecté à `terminer_voyage()` (Clôture le voyage et libère le véhicule).
- **`QPushButton`** (`btn_facturer`) : Connecté à `generer_facture()`.

---

3. Tableau des Voyages (`tableVoyages`)

Le **`QTreeWidget`** affiche l'historique et les voyages en cours. Ses colonnes sont définies comme suit :

1. **ID**
2. **Client**
3. **Départ - Arrivée** (Note: Cette colonne est définie dans l'UI, mais le code Python `load_voyages` sépare actuellement le Départ et l'Arrivée en deux colonnes distinctes dans la base de données. Il y a donc une légère incohérence entre le design (UI) et l'implémentation (Python/SQL) que le code Python a gérée en ajoutant plus de colonnes.)
4. **Date Début**
5. **Distance Estimée (km)**

6. **Véhicule** (Immatriculation)
7. **Statut**

code du fichier main.py : import sqlite3

```
import sys

from PyQt6.QtWidgets import (

    QApplication, QMainWindow, QWidget, QTabWidget, QMessageBox, QVBoxLayout

)

# Note : Les imports QLineEdit, QComboBox, QTreeWidget nécessaires pour la simulation

# des classes de gestion sont omis ici mais sont requis si vous exécutez le code simulé.


# Importez les classes de gestion.

from GestionVoyage import GestionVoyage

from GestionVehicule import GestionVehicule

from GestionReparation import GestionReparation

from GestionStockEtCommandes import GestionStockEtCommandes


# ---------------------------------------------------------------------

# 1. FONCTION DE CRÉATION DE LA BASE DE DONNÉES

# ---------------------------------------------------------------------
```

```
# (La fonction create_agence_tables() est omise ici pour la concision,
# mais elle doit rester celle que vous avez fournie dans le prompt.)

# def create_agence_tables():
#     # ... Le code complet de création des 14 tables (CLIENT, VEHICULE, VOYAGE, REPARATION, etc.)
#     # doit être ici.
#     #### pass                    # NE RIEN FAIRE DU TOUT    dans cette    FONCTION


# ----------------------------------------------------------------------
# 1. FONCTION DE CRÉATION DE LA BASE DE DONNÉES (Non modifiée, mais incluse pour le contexte)
# ----------------------------------------------------------------------

def create_agence_tables():
    """
    Crée toutes les 14 tables de l'agence de transport et de maintenance.
    (Le code SQL complet pour les 14 tables se trouve ici)
```

```
"""

connection = sqlite3.connect('bdd.db')

cursor = connection.cursor()

# cursor.execute('PRAGMA foreign_keys = ON;')



# # --- SIMULATION de l'exécution du script de création des tables ---

# # Pour ne pas surcharger, on assume que les 14 requêtes CREATE TABLE sont ici.

# cursor.execute('CREATE TABLE IF NOT EXISTS CHAUFFEUR (id_chauffeur INTEGER PRIMARY KEY, nom TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS CLIENT (id_client INTEGER PRIMARY KEY, nom TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS TYPE_VEHICULE (id_type INTEGER PRIMARY KEY, nom_type TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS VEHICULE (immatriculation TEXT PRIMARY KEY, id_type INTEGER, FOREIGN KEY(id_type) REFERENCES TYPE_VEHICULE(id_type))')

# cursor.execute('CREATE TABLE IF NOT EXISTS FOURNISSEUR (id_fournisseur INTEGER PRIMARY KEY, nom_fournisseur TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS PIECE (id_piece INTEGER PRIMARY KEY, nom_piece TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS VOYAGE (id_voyage INTEGER PRIMARY KEY, id_client INTEGER, immatriculation TEXT, id_chauffeur INTEGER, statut_voyage TEXT)')

# cursor.execute('CREATE TABLE IF NOT EXISTS FACTURE_CLIENT (id_facture INTEGER PRIMARY KEY, id_voyage INTEGER UNIQUE, FOREIGN KEY(id_voyage) REFERENCES VOYAGE(id_voyage))')

# cursor.execute('CREATE TABLE IF NOT EXISTS REPARATION (id_reparation INTEGER PRIMARY KEY, immatriculation TEXT, type_maintenance TEXT)')
```

```
# cursor.execute('CREATE TABLE IF NOT EXISTS COMMANDE_REPARATION (id_commande INTEGER PRIMARY KEY, id_fournisseur INTEGER)')

# cursor.execute('CREATE TABLE IF NOT EXISTS PASSER (id_reparation INTEGER, id_commande INTEGER, PRIMARY KEY (id_reparation, id_commande))')

# cursor.execute('CREATE TABLE IF NOT EXISTS UTILISER (id_commande INTEGER, id_piece INTEGER, quantite_utilisee INTEGER, PRIMARY KEY (id_commande, id_piece))')

# cursor.execute('CREATE TABLE IF NOT EXISTS PLAN_MAINTENANCE (id_plan INTEGER PRIMARY KEY, id_type INTEGER)')

# cursor.execute('CREATE TABLE IF NOT EXISTS SUIVI_GPS (id_suivi INTEGER PRIMARY KEY, immatriculation TEXT)')

# # --- FIN SIMULATION ---



# Activer la vérification des clés étrangères

cursor.execute('PRAGMA foreign_keys = ON;')


# --- TABLES DE BASE ET DE RÉFÉRENCE ---


# 1. TABLE CHAUFFEUR
```

```
cursor.execute('''
    CREATE TABLE IF NOT EXISTS CHAUFFEUR (
        id_chauffeur INTEGER PRIMARY KEY AUTOINCREMENT,
        nom TEXT NOT NULL,
        prenom TEXT NOT NULL,
        telephone TEXT UNIQUE,
        date_permis DATE,
        type_permis TEXT
    )
''')

# 1. Insertion des données de référence CHAUFFEUR avec INSERT OR IGNORE ---
### INSERT OR IGNORE INTO CHAUFFEUR
try:
    chauffeurs_data = [
        # (Nom, Prénom, Téléphone, Date Permis (YYYY-MM-DD), Type Permis)
        ('Lefevre', 'Marc', '0611223344', '2010-05-15', 'D'),
        ('Dubois', 'Clara', '0655667788', '2018-10-01', 'C'),
        ('Durand', 'Paul', '0799887766', '2005-01-20', 'D')
```

```
    ]

    for nom, prenom, telephone, date_permis, type_permis in chauffeurs_data:
        cursor.execute("""
            INSERT OR IGNORE INTO CHAUFFEUR
            (nom, prenom, telephone, date_permis, type_permis)
            VALUES (?, ?, ?, ?, ?)
        """, (nom, prenom, telephone, date_permis, type_permis))

    connection.commit()
    # print("Données de référence CHAUFFEUR insérées (si elles n'existaient pas déjà).")

except sqlite3.Error as e:
    print(f"Erreur lors de l'insertion des données chauffeur: {e}")
```

```
# 2. TABLE CLIENT
cursor.execute('''
    CREATE TABLE IF NOT EXISTS CLIENT (
        id_client INTEGER PRIMARY KEY AUTOINCREMENT,
        nom TEXT NOT NULL,
        prenom TEXT,
        email TEXT UNIQUE NOT NULL,
        telephone TEXT,
        adresse TEXT
    )
''')

# 2. Insertion des données de référence CLIENT avec INSERT OR IGNORE ---
# Cette étape utilise la colonne 'email' (UNIQUE) pour éviter les doublons.
### INSERT OR IGNORE INTO CLIENT
try:
    clients_data = [
        # (Nom, Prénom, Email, Téléphone, Adresse)
        ('Dupont', 'Jean', 'jean.dupont@client.com', '0123456789', '1 Rue de la Paix'),
```

```
        ('Martin', 'Sophie', 'sophie.martin@client.com', '0987654321', '5 Avenue des Lilas'),
        ('Bernard', 'Alain', 'alain.bernard@client.com', '0600000000', '10 Place de la Liberté')
    ]

    for nom, prenom, email, telephone, adresse in clients_data:
        cursor.execute("""
            INSERT OR IGNORE INTO CLIENT
            (nom, prenom, email, telephone, adresse)
            VALUES (?, ?, ?, ?, ?)
        """, (nom, prenom, email, telephone, adresse))

    connection.commit()
    # Note : Si vous voulez être sûr, vous pouvez ajouter un print ici
    # print("Données de référence CLIENT insérées (si elles n'existaient pas déjà).")

except sqlite3.Error as e:
    print(f"Erreur lors de l'insertion des données client: {e}")
```

```
# 3. TABLE TYPE_VEHICULE
cursor.execute('''
    CREATE TABLE IF NOT EXISTS TYPE_VEHICULE (
        id_type INTEGER PRIMARY KEY AUTOINCREMENT,
        nom_type TEXT NOT NULL UNIQUE,
        intervalle_vidange_km INTEGER,
        intervalle_révision_mois INTEGER
    )
''')

# 3. Insertion des données de référence avec INSERT OR IGNORE
# Si 'nom_type' existe déjà, SQLite IGNORE l'insertion au lieu de lever une erreur.
### INSERT OR IGNORE INTO TYPE_VEHICULE
try:
    data = [
        ('Voiture', 15000, 12),
        ('Camion', 50000, 6),
```

```
        ('Bus', 30000, 9)
    ]

    # Insertion multiple
    for nom, vidange_km, revision_mois in data:
        cursor.execute("""
            INSERT OR IGNORE INTO TYPE_VEHICULE
            (nom_type, intervalle_vidange_km, intervalle_révision_mois)
            VALUES (?, ?, ?)
        """, (nom, vidange_km, revision_mois))

    # conn.commit()
    connection.commit()
    # print("Données de référence TYPE_VEHICULE insérées (si elles n'existaient pas déjà).")

except sqlite3.Error as e:
    print(f"Erreur lors de l'insertion des données de type de véhicule: {e}")
```

```
# 4. TABLE VEHICULE (Régi par TYPE_VEHICULE)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS VEHICULE (
        immatriculation TEXT PRIMARY KEY,
        marque TEXT NOT NULL,
        modele TEXT,
        annee INTEGER,
        kilometrage_actuel INTEGER DEFAULT 0,
        capacite_places INTEGER,
        statut_disponibilite TEXT NOT NULL DEFAULT 'Disponible',
        id_type INTEGER NOT NULL,
        FOREIGN KEY(id_type)
        REFERENCES TYPE_VEHICULE(id_type)
        ON UPDATE CASCADE
        ON DELETE RESTRICT -- Empêche la suppression du type s'il y a des véhicules
    )
```

```
    ''')

    # 4. PRÉPARATION : Récupérer les ID des types de véhicules ---
# --- PRÉPARATION : Récupérer les ID des types de véhicules (Doit être exécuté APRES l'insertion de TYPE_VEHICULE) ---
    cursor.execute("SELECT id_type FROM TYPE_VEHICULE WHERE nom_type = 'Voiture'")
    result_voiture = cursor.fetchone()
    id_type_voiture = result_voiture[0] if result_voiture else None

    cursor.execute("SELECT id_type FROM TYPE_VEHICULE WHERE nom_type = 'Camion'")
    result_camion = cursor.fetchone()
    id_type_camion = result_camion[0] if result_camion else None

    # Vérification critique
    if id_type_voiture is None or id_type_camion is None:
        print("ÉCHEC DE DÉMARRAGE : Impossible de trouver les ID des types 'Voiture' ou 'Camion'. Assurez-vous d'insérer les types de véhicules AVANT les véhicules.")
        return # Arrête l'exécution si les dépendances manquent

    # --- Insertion des données de référence VEHICULE avec INSERT OR IGNORE ---
```

```
try:
    # Assurez-vous que le statut est EXACTEMENT 'Disponible'
    vehicules_data = [
        # (Immat, Marque, Modèle, Année, KM, Places, Statut, ID_Type)
        ('AB-123-CD', 'Renault', 'Clio', 2022, 12000, 5, 'Disponible', id_type_voiture),
        ('EF-456-GH', 'Peugeot', '308', 2021, 50000, 5, 'En course', id_type_voiture),
        ('IJ-789-KL', 'Volvo', 'FH16', 2018, 250000, 2, 'Disponible', id_type_camion),
        ('MN-012-OP', 'Mercedes', 'Actros', 2023, 1000, 2, 'En réparation', id_type_camion)
    ]

    for immat, marque, modele, annee, km, places, statut, id_type in vehicules_data:
        cursor.execute("""
            INSERT OR IGNORE INTO VEHICULE
            (immatriculation, marque, modele, annee, kilometrage_actuel, capacite_places, statut_disponibilite, id_type)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (immat, marque, modele, annee, km, places, statut, id_type))

    connection.commit() # Assurez-vous que ceci est fait
    print("Insertion des véhicules par défaut effectuée.")
```

```
    except sqlite3.Error as e:
        print(f"Erreur lors de l'insertion des données véhicule: {e}")
```

🛠️ Insertion des Données de Référence pour FOURNISSEUR

Comme la colonne nom_fournisseur n'a pas la contrainte UNIQUE, le moyen le plus simple d'assurer le INSERT OR IGNORE sans doublon est de lui ajouter temporairement la contrainte UNIQUE dans le CREATE TABLE OU de simplement faire confiance au mécanisme d'unicité implicite sur l'ensemble de données (en s'assurant de ne pas insérer deux fois le même nom par défaut).

Le code ci-dessous utilise le INSERT OR IGNORE et suppose que l'unicité du nom du fournisseur est une règle métier que nous cherchons à appliquer pour les données par défaut.

Ajoutez ce bloc de code juste après la création de la TABLE FOURNISSEUR (ligne 294) dans votre fonction create_agence_tables() :

```
# 5. TABLE FOURNISSEUR
cursor.execute('''
    CREATE TABLE IF NOT EXISTS FOURNISSEUR (
        id_fournisseur INTEGER PRIMARY KEY AUTOINCREMENT,
        nom_fournisseur TEXT NOT NULL UNIQUE,
        contact TEXT,
        telephone TEXT,
        specialite TEXT -- (Pièces, Garage, Services)
    )
''')

# 5. Insertion des données de référence FOURNISSEUR avec INSERT OR IGNORE ---
# Nous utilisons la colonne 'nom_fournisseur' (maintenant UNIQUE) pour éviter les doublons.
try:
    fournisseurs_data = [
        # (Nom, Contact, Téléphone, Spécialité)
```

```
        ('Pneus Express 2000', 'Marc Dubois', '0140102030', 'Pièces'),
        ('Huiles Lubrifiants SA', 'Sophie Martin', '0450607080', 'Lubrifiants'),
        ('Garage Central Service', 'Jean Bernard', '0911223344', 'Garage/Maintenance')
    ]

    for nom, contact, telephone, specialite in fournisseurs_data:
        cursor.execute("""
            INSERT OR IGNORE INTO FOURNISSEUR
            (nom_fournisseur, contact, telephone, specialite)
            VALUES (?, ?, ?, ?)
        """, (nom, contact, telephone, specialite))

    connection.commit()
    # print("Données de référence FOURNISSEUR insérées.")

  except sqlite3.Error as e:
    print(f"Erreur lors de l'insertion des données fournisseur: {e}")

# Note Importante sur le Schéma
```

J'ai ajouté la contrainte UNIQUE sur la colonne nom_fournisseur dans le CREATE TABLE ci-dessus pour que le INSERT OR IGNORE fonctionne de manière fiable. Si vous ne souhaitez pas que le nom soit unique dans votre modèle de données final, vous devrez implémenter une logique de SELECT avant INSERT en Python pour vérifier l'existence (ce qui est plus complexe). Pour les données par défaut, l'ajout de UNIQUE est la solution standard.

Avec cette modification, la liste déroulante "Fournisseur :" devrait maintenant être remplie au démarrage de votre application.

Voulez-vous que je vérifie un autre onglet ou une autre fonctionnalité maintenant que les données de base sont complètes ?

```
# 6. TABLE PIECE (Sourcée par FOURNISSEUR)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS PIECE (
        id_piece INTEGER PRIMARY KEY AUTOINCREMENT,
        nom_piece TEXT NOT NULL,
```

```
        reference_fournisseur TEXT UNIQUE,
        prix_unitaire REAL NOT NULL,
        quantite_stock INTEGER DEFAULT 0
        -- Note: La relation SOURCER (M-N) est gérée plus loin si nécessaire,
        -- ici on se concentre sur les relations 1-N directes.
    )
''')

# --- TABLES DES OPÉRATIONS (VOYAGES ET MAINTENANCE) ---

# # 7. TABLE VOYAGE (Réservé par CLIENT, Attribué à VEHICULE et CHAUFFEUR)
# cursor.execute('''
#     CREATE TABLE IF NOT EXISTS VOYAGE (
#         id_voyage INTEGER PRIMARY KEY AUTOINCREMENT,
#         date_debut DATETIME NOT NULL,
#         date_fin DATETIME,
#         lieu_depart TEXT NOT NULL,
#         lieu_arrivee TEXT NOT NULL,
#         distance_estimee INTEGER,
```

```
#        statut_voyage TEXT NOT NULL DEFAULT 'Planifié',
#        id_client INTEGER NOT NULL,
#        immatriculation TEXT NOT NULL,
#        id_chauffeur INTEGER NOT NULL,
#        FOREIGN KEY(id_client) REFERENCES CLIENT(id_client)
#            ON UPDATE CASCADE ON DELETE RESTRICT,
#        FOREIGN KEY(immatriculation) REFERENCES VEHICULE(immatriculation)
#            ON UPDATE CASCADE ON DELETE RESTRICT,
#        FOREIGN KEY(id_chauffeur) REFERENCES CHAUFFEUR(id_chauffeur)
#            ON UPDATE CASCADE ON DELETE RESTRICT
#    )
# ''')



# 7. TABLE VOYAGE (Réservé par CLIENT, Attribué à VEHICULE et CHAUFFEUR)
#########
# CREATE TABLE IF NOT EXISTS VOYAGE (
#    id_voyage            INTEGER PRIMARY KEY AUTOINCREMENT,
#    id_client            INTEGER NOT NULL,
```

```
#    immatriculation      TEXT NOT NULL,
#    id_chauffeur         INTEGER NOT NULL,

#    # Détails du voyage
#    lieu_depart          TEXT NOT NULL,
#    lieu_arrivee         TEXT NOT NULL,

#    # Dates
#    date_debut_planifiee  TEXT,            # Format 'YYYY-MM-DD HH:MM:SS'
#    date_fin_reelle       TEXT,            # Date d'arrivée réelle (ajoutée pour corriger l'erreur précédente)

#    # État du voyage
#    statut_voyage         TEXT DEFAULT 'Planifié', # Ex: 'Planifié', 'En course', 'Terminé', 'Facturé'

#    # Clés étrangères (assurez-vous que les tables CLIENT, VEHICULE, CHAUFFEUR existent)
#    FOREIGN KEY (id_client) REFERENCES CLIENT(id_client),
#    FOREIGN KEY (immatriculation) REFERENCES VEHICULE(immatriculation),
#    FOREIGN KEY (id_chauffeur) REFERENCES CHAUFFEUR(id_chauffeur)
# );
```

```
#########
cursor.execute('''
    CREATE TABLE IF NOT EXISTS VOYAGE (
        id_voyage INTEGER PRIMARY KEY AUTOINCREMENT,
        date_debut_planifiee DATETIME NOT NULL,
        date_fin_reelle DATETIME,
        lieu_depart TEXT NOT NULL,
        lieu_arrivee TEXT NOT NULL,
        distance_estimee INTEGER,
        statut_voyage TEXT NOT NULL DEFAULT 'Planifié',
        id_client INTEGER NOT NULL,
        immatriculation TEXT NOT NULL,
        id_chauffeur INTEGER NOT NULL,
        FOREIGN KEY(id_client) REFERENCES CLIENT(id_client)
            ON UPDATE CASCADE ON DELETE RESTRICT,
        FOREIGN KEY(immatriculation) REFERENCES VEHICULE(immatriculation)
            ON UPDATE CASCADE ON DELETE RESTRICT,
        FOREIGN KEY(id_chauffeur) REFERENCES CHAUFFEUR(id_chauffeur)
            ON UPDATE CASCADE ON DELETE RESTRICT
```

```
    )
''')



# 8. TABLE FACTURE_CLIENT (Liée à VOYAGE)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS FACTURE_CLIENT (
        id_facture INTEGER PRIMARY KEY AUTOINCREMENT,
        date_emission DATE NOT NULL,
        date_echeance DATE,
        montant_HT REAL NOT NULL,
        montant_TVA REAL,
        statut_paiement TEXT NOT NULL DEFAULT 'En attente',
        id_voyage INTEGER UNIQUE NOT NULL, -- Un voyage = une facture
        FOREIGN KEY(id_voyage) REFERENCES VOYAGE(id_voyage)
            ON UPDATE CASCADE ON DELETE RESTRICT
    )
''')
```

```
   # 9. TABLE REPARATION (Concerne VEHICULE, Peut être Sous-Traitée à FOURNISSEUR)
# CREATE TABLE IF NOT EXISTS REPARATION (
#     id_reparation        INTEGER PRIMARY KEY AUTOINCREMENT,
#     immatriculation      TEXT NOT NULL,
#     type_maintenance     TEXT NOT NULL,    -- Ex: 'Corrective' ou 'Préventive'
#     nature_panne_tache   TEXT,             -- Description de la panne ou de la tâche effectuée
#     date_debut           TEXT NOT NULL,    -- Format 'YYYY-MM-DD'
#     date_fin_reelle      TEXT,             -- Date de fin de l'intervention (peut être NULL tant que la réparation est en cours)
#     cout_main_oeuvre     REAL,             -- Coût de la main-d'œuvre (hors pièces)
#     statut_reparation    TEXT DEFAULT 'En cours', -- Ex: 'En cours', 'Terminée', 'Facturée'

#     # Relation avec les Véhicules
#     FOREIGN KEY (immatriculation) REFERENCES VEHICULE(immatriculation)
# );

   # CREATE TABLE IF NOT EXISTS REPARATION (
   #     id_reparation           INTEGER PRIMARY KEY AUTOINCREMENT,
   #     date_debut_reparation   TEXT NOT NULL,    -- Correspond à 'date_debut' dans le code Python
```

```
#    date_fin_reparation      TEXT,            -- Correspond à 'date_fin_reelle' dans le code Python, NULL si en cours
#    cout_main_d_oeuvre       REAL,            -- Correspond à 'cout_main_oeuvre' dans le code Python
#    nature_panne_tache       TEXT,
#    description_travaux      TEXT,
#    type_maintenance         TEXT NOT NULL,    -- Ex: 'Préventive' ou 'Corrective'
#    statut_reparation        TEXT DEFAULT 'En cours', -- Ajouté pour le suivi
#    immatriculation          TEXT NOT NULL,
#    id_fournisseur           INTEGER,         -- NULL si réparation interne

#    FOREIGN KEY(immatriculation) REFERENCES VEHICULE(immatriculation)
#        ON UPDATE CASCADE ON DELETE RESTRICT,
#    FOREIGN KEY(id_fournisseur) REFERENCES FOURNISSEUR(id_fournisseur)
#        ON UPDATE CASCADE ON DELETE SET NULL
# );

cursor.execute('''
    CREATE TABLE IF NOT EXISTS REPARATION (
        id_reparation INTEGER PRIMARY KEY AUTOINCREMENT,
        date_debut_reparation DATETIME NOT NULL,
```

```
        date_fin_reparation DATETIME,
        cout_main_d_oeuvre      REAL,
        nature_panne_tache TEXT,
        description_travaux TEXT,
        type_maintenance TEXT NOT NULL, -- Préventive ou Corrective
        statut_reparation        TEXT DEFAULT 'En cours', -- Ajouté pour le suivi
        immatriculation TEXT NOT NULL,
        id_fournisseur INTEGER, -- NULL si réparation interne
        FOREIGN KEY(immatriculation) REFERENCES VEHICULE(immatriculation)
            ON UPDATE CASCADE ON DELETE RESTRICT,
        FOREIGN KEY(id_fournisseur) REFERENCES FOURNISSEUR(id_fournisseur)
            ON UPDATE CASCADE ON DELETE SET NULL
    )
''')



# 10. TABLE COMMANDE_REPARATION (Liée à REPARATION et à FOURNISSEUR)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS COMMANDE_REPARATION (
```

```
        id_commande INTEGER PRIMARY KEY AUTOINCREMENT,
        date_commande DATE NOT NULL,
        montant_estime REAL,
        statut_commande TEXT NOT NULL DEFAULT 'En cours',
        id_fournisseur INTEGER NOT NULL,
        FOREIGN KEY(id_fournisseur) REFERENCES FOURNISSEUR(id_fournisseur)
            ON UPDATE CASCADE ON DELETE RESTRICT
    )
''')

# --- TABLES DE LIAISON (Relations M:N) ---

# 11. TABLE PASSER (Liaison M:N entre REPARATION et COMMANDE_REPARATION)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS PASSER (
        id_reparation INTEGER NOT NULL,
        id_commande INTEGER NOT NULL,
        PRIMARY KEY (id_reparation, id_commande),
        FOREIGN KEY(id_reparation) REFERENCES REPARATION(id_reparation)
```

```
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY(id_commande) REFERENCES COMMANDE_REPARATION(id_commande)
        ON UPDATE CASCADE ON DELETE CASCADE
  )
''')


# 12. TABLE UTILISER (Liaison M:N entre COMMANDE_REPARATION et PIECE)
cursor.execute('''
  CREATE TABLE IF NOT EXISTS UTILISER (
    id_commande INTEGER NOT NULL,
    id_piece INTEGER NOT NULL,
    quantite_utilisee INTEGER NOT NULL,
    PRIMARY KEY (id_commande, id_piece),
    FOREIGN KEY(id_commande) REFERENCES COMMANDE_REPARATION(id_commande)
        ON UPDATE CASCADE ON DELETE CASCADE,
    FOREIGN KEY(id_piece) REFERENCES PIECE(id_piece)
        ON UPDATE CASCADE ON DELETE RESTRICT
  )
''')
```

```
# --- TABLES DE MAINTENANCE ET GPS ---


# 13. TABLE PLAN_MAINTENANCE (Définie par TYPE_VEHICULE)
cursor.execute('''
    CREATE TABLE IF NOT EXISTS PLAN_MAINTENANCE (
        id_plan INTEGER PRIMARY KEY AUTOINCREMENT,
        description_tache TEXT NOT NULL,
        frequence_km INTEGER, -- NULL si basée sur le temps
        frequence_jours INTEGER, -- NULL si basée sur le kilométrage
        est_active BOOLEAN NOT NULL DEFAULT 1,
        id_type INTEGER NOT NULL,
        FOREIGN KEY(id_type) REFERENCES TYPE_VEHICULE(id_type)
            ON UPDATE CASCADE ON DELETE CASCADE
    )
''')


# 14. TABLE SUIVI_GPS (Localise VEHICULE)
cursor.execute('''
```

```
    CREATE TABLE IF NOT EXISTS SUIVI_GPS (
        id_suivi INTEGER PRIMARY KEY AUTOINCREMENT,
        date_heure DATETIME NOT NULL,
        latitude REAL NOT NULL,
        longitude REAL NOT NULL,
        vitesse INTEGER,
        immatriculation TEXT NOT NULL,
        FOREIGN KEY(immatriculation) REFERENCES VEHICULE(immatriculation)
            ON UPDATE CASCADE ON DELETE CASCADE
    )
''')


connection.commit()
connection.close()


# ----------------------------------------------------------------------
```

```
# 2. CLASSE PRINCIPALE DE L'APPLICATION PyQt6
# ----------------------------------------------------------------------


class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Système de Gestion d'Agence de Transport")
        self.setGeometry(100, 100, 1400, 850)

        try:
            create_agence_tables()
        except Exception as e:
            QMessageBox.critical(self, "Erreur DB", f"Erreur fatale lors de la création des tables: {e}")
            sys.exit(1)

        self.central_widget = QWidget()
        self.setCentralWidget(self.central_widget)

        self.table = QTabWidget()
```

```
    self.table.setTabPosition(QTabWidget.TabPosition.North)

    main_layout = QVBoxLayout(self.central_widget)
    main_layout.addWidget(self.table)

    self.setup_modules()

def db_connect(self):
    """Établit et retourne une connexion à la base de données SQLite."""
    conn = sqlite3.connect('bdd.db')
    conn.execute('PRAGMA foreign_keys = ON;')
    return conn

def setup_modules(self):
    """Initialise chaque classe de gestion et établit les connexions de signaux."""

    try:
        self.gestion_voyage = GestionVoyage(self.db_connect)
        self.gestion_vehicule = GestionVehicule(self.db_connect)
```

```
self.gestion_reparation = GestionReparation(self.db_connect)
self.gestion_stock = GestionStockEtCommandes(self.db_connect) # Assurez-vous que cette classe existe

self.table.addTab(self.gestion_voyage, "✈️ Gestion des Voyages")
self.table.addTab(self.gestion_vehicule, "🚚 Gestion des Véhicules")
self.table.addTab(self.gestion_reparation, "🔧 Gestion des Réparations")
self.table.addTab(self.gestion_stock, "📦 Stock & Commandes")

# 🎯 CONNEXIONS BIDIRECTIONNELLES CENTRALISÉES 🎯

# --- ÉVÉNEMENT 1 : Changement de statut de Véhicule (Ajout/Modif/Suppr. MANUELLE)
# Émetteur : GestionVehicule.vehicule_modifie
# Récepteurs : Tous les modules qui listent les véhicules disponibles
self.gestion_vehicule.vehicule_modifie.connect(self.gestion_voyage.charger_comboboxes)
self.gestion_vehicule.vehicule_modifie.connect(self.gestion_reparation.charger_vehicules_a_reparer)

# --- ÉVÉNEMENT 2 : Fin de Voyage (Statut: En course -> Disponible)
# Émetteur : GestionVoyage.voyage_termine
```

```
        # Récepteurs : Le tableau des véhicules (affichage) et le module de réparation (car les véhicules sont maintenant disponibles)
        self.gestion_voyage.voyage_termine.connect(self.gestion_vehicule.load_vehicules)
        self.gestion_voyage.voyage_termine.connect(self.gestion_reparation.charger_vehicules_a_reparer)

        # --- ÉVÉNEMENT 3 : Fin de Réparation (Statut: En réparation -> Disponible)
        # Émetteur : GestionReparation.reparation_terminee
        # Récepteurs : Le tableau des véhicules (affichage) et le module de voyage (liste des véhicules disponibles)
        self.gestion_reparation.reparation_terminee.connect(self.gestion_vehicule.load_vehicules)
        self.gestion_reparation.reparation_terminee.connect(self.gestion_voyage.charger_comboboxes)

        # --- Connexion de rafraîchissement au changement d'onglet (Fallback) ---
        self.table.currentChanged.connect(self.rafraichir_onglet_actif)

    except Exception as e:
        QMessageBox.critical(self, "Erreur d'Initialisation", f"Impossible de charger un ou plusieurs modules : {e}")
        sys.exit(1)


def rafraichir_onglet_actif(self, index_selectionne):
```

```
"""Appelle la méthode de rafraîchissement appropriée selon l'onglet activé."""

onglet_actif = self.table.widget(index_selectionne)

if onglet_actif == self.gestion_vehicule:
    self.gestion_vehicule.load_vehicules()

elif onglet_actif == self.gestion_voyage:
    self.gestion_voyage.load_voyages()
    self.gestion_voyage.charger_comboboxes() # Rafraîchissement des listes (Véhicules dispo, etc.)

elif onglet_actif == self.gestion_reparation:
    self.gestion_reparation.load_reparations()
    self.gestion_reparation.charger_vehicules_a_reparer()

elif onglet_actif == self.gestion_stock:
    # Assurez-vous que GestionStockEtCommandes a une méthode load_data()
    # self.gestion_stock.load_data()
    pass
```

```
# ----------------------------------------------------------------------

# 3. POINT D'ENTRÉE (Exécution)

# ----------------------------------------------------------------------



if __name__ == '__main__':

    app = QApplication(sys.argv)

    main_window = MainWindow()

    main_window.show()

    sys.exit(app.exec())
```

Le fichier `main.py` est le **point d'entrée** de l'application. Il est responsable de la création de la base de données, de l'initialisation de l'interface principale (`MainWindow`), de l'intégration des différents modules de gestion (`GestionVoyage`, `GestionVehicule`, etc.) sous forme d'onglets, et surtout, de l'établissement des **connexions de signaux** nécessaires à la synchronisation des données entre ces modules.

---

1. Fonction `create_agence_tables()`

Cette fonction exécute les requêtes SQL pour créer les **14 tables** de la base de données SQLite (`bdd.db`) et y insère des **données de référence** (chauffeurs, clients, types de véhicules, véhicules, fournisseurs).

Table	Rôle Principal	Clés Étrangères (Dépendances)	Données Insérées par Défaut
`VEHICULE`	Gère les actifs de transport. Contient le `statut_disponibilite`.	`TYPE_VEHICULE`	4 véhicules avec différents statuts (Disponible, En course, En réparation).

Table	Rôle Principal	Clés Étrangères (Dépendances)	Données Insérées par Défaut
`VOYAGE`	Planifie les trajets et leur statut (`Planifié`, `Terminé`, `Facturé`).	`CLIENT`, `VEHICULE`, `CHAUFFEUR`	Aucune insertion, l'utilisateur doit les créer.
`CHAUFFEUR`	Gère les informations des conducteurs.	(Aucune)	3 chauffeurs.
`CLIENT`	Gère les informations des clients.	(Aucune)	3 clients.
`TYPE_VEHICULE`	Définit les règles de maintenance par catégorie.	(Aucune)	Voiture, Camion, Bus.
`FOURNISSEUR`	Gère les prestataires externes.	(Aucune)	3 fournisseurs.
`PIECE`	Gère le stock de pièces détachées.	(Aucune)	Aucune, stock vide initial.
Autres Tables	`FACTURE_CLIENT`, `REPARATION`, `COMMANDE_REPARATION`, `PASSER`, `UTILISER`, `PLAN_MAINTENANCE`, `SUIVI_GPS`.	Maintiennent les relations N:M et le suivi des opérations.	Aucune insertion pour les tables opérationnelles.

---

2. Classe `MainWindow` (Application PyQt6)

Initialisation

1. La fonction `create_agence_tables()` est appelée pour s'assurer que la base de données et les données de référence sont prêtes.
2. Un **`QTabWidget`** (`self.table`) est créé pour organiser l'interface en onglets.
3. La méthode `setup_modules()` est appelée pour initialiser les classes de gestion.

Connexion DB (`db_connect`)

La méthode `db_connect()` est essentielle : elle garantit que chaque module peut établir une **connexion SQLite indépendante** à `bdd.db` et que les **clés étrangères** (`PRAGMA foreign_keys = ON;`) sont activées pour chaque connexion.

Gestion des Signaux Centralisée (`setup_modules`) 🎯

C'est la partie la plus importante pour l'interopérabilité de l'application. Elle assure la **synchronisation immédiate** des données entre les différents onglets lorsqu'un changement a lieu dans la base de données.

Événement	Émetteur (`Signal`)	Récepteur 1	Récepteur 2
Véhicule Modifié (Manuel)	`GestionVehicule.vehicule_modifie`	`GestionVoyage.charger_comboboxes` (Rafraîchit la liste des véhicules dispos)	`GestionReparation.charger_vehicules_a_reparer`

Événement	Émetteur (`Signal`)	Récepteur 1	Récepteur 2
Fin de Voyage (`Terminer`)	`GestionVoyage.voyage_termine`	`GestionVehicule.load_vehicules` (Rafraîchit le tableau des véhicules)	`GestionReparation.charger_vehicules_a_reparer`
Fin de Réparation (`Terminée`)	`GestionReparation.reparation_terminee`	`GestionVehicule.load_vehicules` (Rafraîchit le tableau des véhicules)	`GestionVoyage.charger_comboboxes` (Rafraîchit la liste des véhicules dispos)

De plus, l'événement **`self.table.currentChanged.connect(self.rafraichir_onglet_actif)`** fournit un mécanisme de **rafraîchissement par défaut** lorsque l'utilisateur change d'onglet, garantissant une cohérence visuelle même si un signal a été manqué ou si une mise à jour croisée est nécessaire.

1) Voici l’erreur apparu : Impossible de

générer la facture : NOT NULL constraint failed :

FACTURE_CLIENT.date_emission

2) Comment remplir le CHAMP nommé

"date_emission" contenu dans la TABLE nommée

"FACTURE_CLIENT" ?

Est ce que le CHAMP nommé "date_emission" devrait

t il contenir la même valeur que celle de la COLONNE "Date Fin

Réelle" contenu dans la QTreeWidget nommée "tableVoyages"

contenu ans l'onglet "Gestion des Voyages" ?

3) Comment également remplir tous les autres champs

nommés "date_echeance" , "montant_HT" ,

"montant_TVA" , "statut_paiement" contenu dans la TABLE

"FACTURE_CLIENT" ?

L'erreur `NOT NULL constraint failed: FACTURE_CLIENT.date_emission` se produit car la colonne `date_emission` est définie comme **non nulle** (`NOT NULL`) dans votre schéma SQL (voir `main.py`), mais la méthode `generer_facture` dans `GestionVoyage.py` ne lui fournit aucune valeur.

Voici comment résoudre ce problème et gérer les autres champs de la table `FACTURE_CLIENT`.

---

1. Correction de l'Erreur `NOT NULL` pour `date_emission`

Pour résoudre cette erreur, vous devez fournir une valeur pour `date_emission` lors de l'insertion dans la table `FACTURE_CLIENT`. La **date d'émission** d'une facture est généralement la **date du jour** où la facture est générée.

Modifiez la méthode `generer_facture(self)` dans `GestionVoyage.py` comme suit :

Python
```
# Dans GestionVoyage.py, méthode generer_facture(self)

def generer_facture(self):
    # ... (code de vérification omis) ...

    try:
        conn = self.db_connect()
        cursor = conn.cursor()

        # ... (Vérifier si une facture existe déjà) ...

        # 🎯 AJOUT 1 : Déterminer la date d'émission (date du jour)
        date_emission = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

        # 🎯 AJOUT 2 : Déterminer les autres valeurs par défaut pour l'insertion
        # Pour le moment, nous utilisons des valeurs par défaut/temporaires (voir section 3)
        # Ces valeurs doivent être calculées si votre application le permet.
        montant_ht = 0.00  # Valeur temporaire
        montant_tva = 0.00 # Valeur temporaire
        statut_paiement = 'En attente' # Valeur par défaut
```

```
        # Insérer la nouvelle facture (maintenant avec date_emission et autres champs)
        cursor.execute("""
            INSERT INTO FACTURE_CLIENT (id_voyage, date_emission, montant_HT, montant_TVA, statut_paiement)
            VALUES (?, ?, ?, ?, ?)
        """, (id_voyage, date_emission, montant_ht, montant_tva, statut_paiement))

        # ... (Mettre à jour le statut du voyage à 'Facturé') ...

        conn.commit()
        # ... (Message de succès et rafraîchissement) ...

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de générer la facture: {e}")
    finally:
        # ... (conn.close()) ...
```

---

2. Remplissage du Champ `date_emission` et Cohérence avec le Tableau

Comment remplir le CHAMP `date_emission` ?

Le champ `date_emission` doit être rempli avec la **date et l'heure actuelles** au moment où l'utilisateur clique sur le bouton "Générer Facture".

Python
```
date_emission = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
```

Cohérence avec `Date Fin Réelle` ?

Non, le champ `date_emission` ne doit **pas** nécessairement contenir la même valeur que la colonne `Date Fin Réelle` du voyage.

- **`Date Fin Réelle`** : C'est la date de la **fin du service** (la clôture effective du voyage).
- **`date_emission`** : C'est la date de l'**émission administrative** de la facture.

Il est courant qu'une facture soit émise quelques heures, jours, voire semaines *après* la fin du service (le voyage). Dans ce système simple, utiliser la date actuelle (`datetime.now()`) pour la date d'émission est la pratique la plus logique.

---

3. Remplissage des Autres Champs de `FACTURE_CLIENT`

Les champs `date_echeance`, `montant_HT`, `montant_TVA` et `statut_paiement` nécessitent une logique métier pour être remplis correctement.

a) `statut_paiement` (Facile)

Ce champ a une valeur par défaut dans votre schéma SQL (`'En attente'`). Vous pouvez utiliser cette valeur par défaut lors de l'insertion, ou l'insérer explicitement :

- **Valeur par défaut à l'insertion :** `'En attente'`

b) `date_echeance` (Règle métier)

La date d'échéance est généralement la `date_emission` plus un délai de paiement standard (ex: 30 jours, 60 jours).

- **Logique :** Date Eche´ance=Date Emission+De´lai de Paiement

Python
```
from datetime import datetime, timedelta

# ... dans generer_facture ...
date_emission = datetime.now() # Ne pas formater en string tout de suite
date_echeance = date_emission + timedelta(days=30) # Ajout d'un délai de 30 jours

# Formatage pour la base de données
date_emission_str = date_emission.strftime('%Y-%m-%d %H:%M:%S')
date_echeance_str = date_echeance.strftime('%Y-%m-%d %H:%M:%S')
```

c) `montant_HT` et `montant_TVA` (Calcul)

Ces montants doivent être calculés à partir des données du voyage sélectionné.

- **Base de calcul :** Le coût total est généralement basé sur la **distance parcourue** (ou estimée) et un **tarif unitaire** (ex: Tarif au km).

1. **Récupérer les données du voyage :** Vous devez récupérer le `distance_estimee` du voyage à facturer.
2. **Définir le tarif :** Si vous utilisez un tarif fixe, par exemple 2.50 €/km (HT) :
   - Montant HT=Distance Estime´e×Tarif au km (HT)
   - Montant TVA=Montant HT×Taux TVA (ex: 0.20)

Vous devrez donc effectuer une requête **`SELECT`** pour obtenir le `distance_estimee` avant l'**`INSERT`** de la facture.

Python

```
# ... dans generer_facture ...

# 1. Récupération des données du voyage pour le calcul
cursor.execute("SELECT distance_estimee FROM VOYAGE WHERE id_voyage = ?", (id_voyage,))
result = cursor.fetchone()

if result:
    distance_estimee = result[0] if result[0] is not None else 0
else:
    # Gérer le cas où le voyage n'existe pas (devrait être impossible ici)
    distance_estimee = 0

# 2. Définition des règles de facturation
TARIF_KM_HT = 2.50 # Exemple
TAUX_TVA = 0.20  # 20%

# 3. Calculs
montant_ht = round(distance_estimee * TARIF_KM_HT, 2)
montant_tva = round(montant_ht * TAUX_TVA, 2)
```

En utilisant ces logiques, vous pouvez insérer les valeurs correctes et complètes dans la table `FACTURE_CLIENT`.

code du fichier GestionVoyage.py : import sqlite3

from PyQt6.QtCore import pyqtSignal

from PyQt6.QtWidgets import QWidget, QMessageBox, QTreeWidgetItem

from PyQt6.uic import loadUi

from datetime import datetime

class GestionVoyage(QWidget):

SIGNAL : Émis lorsque le statut d'un véhicule change suite à la fin d'un voyage.

```
voyage_termine = pyqtSignal()

def __init__(self, db_connect_func):
    super().__init__()
    # Chargement de l'interface utilisateur
    loadUi("GestionVoyage.ui", self)

    self.db_connect = db_connect_func

    # Connexion des boutons
    self.btn_planifier.clicked.connect(self.planifier_voyage)
    self.btn_terminer.clicked.connect(self.terminer_voyage)
    self.btn_facturer.clicked.connect(self.generer_facture)

    # Connexion de l'événement de sélection pour remplir les champs
    self.tableVoyages.itemSelectionChanged.connect(self.charger_champs_voyage)

    # Initialisation des données
    self.charger_comboboxes()
```

```
    self.load_voyages()

def charger_comboboxes(self):
    """[SLOT] Charge les listes déroulantes (clients, chauffeurs, véhicules disponibles)."""
    try:
        conn = self.db_connect()
        cursor = conn.cursor()

        # --- Clients (avec ID stocké) ---
        cursor.execute("SELECT id_client, nom || ' ' || prenom FROM CLIENT ORDER BY nom, prenom")
        clients = cursor.fetchall()
        self.combo_client.clear()
        self.combo_client.addItem("Sélectionner un client...", -1)

        for id_client, nom_complet in clients:
            self.combo_client.addItem(nom_complet, id_client)

        # --- Chauffeurs (avec ID stocké) ---
        cursor.execute("SELECT id_chauffeur, nom || ' ' || prenom FROM CHAUFFEUR ORDER BY nom, prenom")
```

```
    chauffeurs = cursor.fetchall()
    self.combo_chauffeur.clear()
    self.combo_chauffeur.addItem("Sélectionner un chauffeur...", -1)

    for id_chauffeur, nom_complet in chauffeurs:
        self.combo_chauffeur.addItem(nom_complet, id_chauffeur)

    # --- Véhicules (uniquement Disponibles) ---
    cursor.execute("SELECT immatriculation FROM VEHICULE WHERE statut_disponibilite = 'Disponible'")
    vehicules = [row[0] for row in cursor.fetchall()]
    self.combo_vehicule.clear()
    self.combo_vehicule.addItem("Sélectionner un véhicule disponible...")
    self.combo_vehicule.addItems(vehicules)

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des ComboBoxes: {e}")
finally:
    if 'conn' in locals() and conn:
        conn.close()
```

```
# def load_voyages(self):
#     """[SLOT] Charge tous les voyages dans le QTreeWidget."""
#     try:
#         conn = self.db_connect()
#         cursor = conn.cursor()
#         # Note : distance_estimee n'est pas affichée ici, mais peut être ajoutée au SELECT/setHeaderLabels si nécessaire.
#         cursor.execute("""
#             SELECT
#                 V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
#                 V.lieu_depart, V.lieu_arrivee, V.date_fin_reelle
#             FROM VOYAGE V
#             JOIN CLIENT C ON V.id_client = C.id_client
#             JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
#             JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
#             ORDER BY V.id_voyage DESC
#         """)
#         voyages = cursor.fetchall()
```

```
#        self.tableVoyages.clear()

#        self.tableVoyages.setHeaderLabels([
#            "ID Voyage", "Client", "Chauffeur", "Immatriculation",
#            "Statut", "Départ", "Arrivée", "Date Fin"
#        ])

#        for row_data in voyages:
#            item = QTreeWidgetItem(self.tableVoyages)
#            for col_num, data in enumerate(row_data):
#                item.setText(col_num, str(data if data is not None else ''))

#    except sqlite3.Error as e:
#        QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
#    finally:
#        if 'conn' in locals() and conn:
#            conn.close()

# def load_voyages(self):
```

```
#        """[SLOT] Charge tous les voyages dans le QTreeWidget."""
#        try:
#            conn = self.db_connect()
#            cursor = conn.cursor()

#            # --- MODIFICATION ICI : Ajout de V.date_debut_planifiee ---
#            cursor.execute("""
#                SELECT
#                    V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
#                    V.lieu_depart, V.lieu_arrivee, V.date_debut_planifiee, V.date_fin_reelle
#                FROM VOYAGE V
#                JOIN CLIENT C ON V.id_client = C.id_client
#                JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
#                JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
#                ORDER BY V.id_voyage DESC
#            """)
#            voyages = cursor.fetchall()

#            self.tableVoyages.clear()
```

```
#           # --- MODIFICATION CI-DESSOUS : Ajustement du header ---
#           self.tableVoyages.setHeaderLabels([
#               "ID Voyage", "Client", "Chauffeur", "Immatriculation",
#               "Statut", "Départ", "Arrivée", "Date Début", "Date Fin" # Nouvelle colonne "Date Début"
#           ])

#           for row_data in voyages:
#               item = QTreeWidgetItem(self.tableVoyages)
#               for col_num, data in enumerate(row_data):
#                   item.setText(col_num, str(data if data is not None else ''))

#       except sqlite3.Error as e:
#           QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
#       finally:
#           if 'conn' in locals() and conn:
#               conn.close()

def load_voyages(self):
```

```
"""[SLOT] Charge tous les voyages dans le QTreeWidget."""
try:
    conn = self.db_connect()
    cursor = conn.cursor()

    # ATTENTION : La requête originale n'a que 8 colonnes.
    # Nous devons l'ajuster pour inclure la distance estimée et la date de début (si elle est nécessaire, basé sur l'UI).
    # Nous utiliserons 9 colonnes dans cet exemple pour inclure toutes les données pertinentes.

    cursor.execute("""
        SELECT
            V.id_voyage, C.nom, Ch.nom, Ve.immatriculation, V.statut_voyage,
            V.lieu_depart, V.lieu_arrivee, V.date_debut_planifiee, V.distance_estimee, V.date_fin_reelle
        FROM VOYAGE V
        JOIN CLIENT C ON V.id_client = C.id_client
        JOIN CHAUFFEUR Ch ON V.id_chauffeur = Ch.id_chauffeur
        JOIN VEHICULE Ve ON V.immatriculation = Ve.immatriculation
        ORDER BY V.id_voyage DESC
    """)
```

```
    voyages = cursor.fetchall()

    self.tableVoyages.clear()

    # --- ÉTAPE 2 : Mise à jour des en-têtes de colonnes (10 colonnes au total) ---
    # L'ordre doit correspondre à la requête SQL.
    self.tableVoyages.setHeaderLabels([
        "ID Voyage", "Client", "Chauffeur", "Immatriculation", "Statut",
        "Départ", "Arrivée", "Date Début", "Distance Estimée (km)", "Date Fin Réelle"
    ])

    # --- ÉTAPE 3 : Remplissage du QTreeWidget ---
    for row_data in voyages:
        item = QTreeWidgetItem(self.tableVoyages)
        for col_num, data in enumerate(row_data):
            item.setText(col_num, str(data if data is not None else ''))

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur DB", f"Erreur lors du chargement des voyages: {e}")
```

```
        finally:
            if 'conn' in locals() and conn:
                conn.close()


def charger_champs_voyage(self):
    """Remplit les champs du formulaire avec les données du voyage sélectionné."""
    selected_item = self.tableVoyages.currentItem()
    if selected_item:
        try:
            # La colonne distance_estimee n'est pas directement affichée dans le QTreeWidget actuel
            # On ne peut donc pas la recharger directement sans modifier la requête load_voyages

            self.combo_client.setCurrentText(selected_item.text(1))
            self.combo_chauffeur.setCurrentText(selected_item.text(2))
        except:
            pass



def planifier_voyage(self):
```

```
"""Enregistre un nouveau voyage, inclut la distance estimée et met le véhicule 'En course'."""

# Récupération des IDs
id_client = self.combo_client.currentData()
id_chauffeur = self.combo_chauffeur.currentData()

immatriculation = self.combo_vehicule.currentText()
depart = self.input_depart.text().strip()
arrivee = self.input_arrivee.text().strip()
date_debut = self.date_debut.dateTime().toString("yyyy-MM-dd HH:mm:ss")

# 🎯 AJOUT / CORRECTION : Lecture et validation de la distance estimée
distance_input = self.input_distance_estimee.text().strip()

try:
    # Si le champ est vide, distance_estimee sera 0.0. Sinon, on tente de convertir.
    distance_estimee = float(distance_input) if distance_input else 0.0
    if distance_estimee < 0:
        QMessageBox.warning(self, "Erreur de Saisie", "La distance estimée ne peut pas être négative.")
```

```
        return
except ValueError:
    QMessageBox.warning(self, "Erreur de Saisie", "La distance estimée doit être un nombre valide.")
    return

if id_client is None or id_client == -1:
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez sélectionner un Client valide.")
    return

if id_chauffeur is None or id_chauffeur == -1:
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez sélectionner un Chauffeur valide.")
    return

if not all([immatriculation, depart, arrivee]) or immatriculation == "Sélectionner un véhicule disponible...":
    QMessageBox.warning(self, "Erreur de Saisie", "Veuillez remplir l'immatriculation, le lieu de départ et d'arrivée.")
    return

try:
    conn = self.db_connect()
```

```
cursor = conn.cursor()

# 🎯 CORRECTION : Mise à jour de la requête INSERT pour inclure distance_estimee
cursor.execute("""
    INSERT INTO VOYAGE (id_client, immatriculation, id_chauffeur, statut_voyage,
                        lieu_depart, lieu_arrivee, date_debut_planifiee, distance_estimee)
    VALUES (?, ?, ?, 'Planifié', ?, ?, ?, ?)
""", (id_client, immatriculation, id_chauffeur, depart, arrivee, date_debut, distance_estimee))

# Mettre le statut du VEHICULE à 'En course'
cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'En course' WHERE immatriculation = ?", (immatriculation,))

conn.commit()
QMessageBox.information(self, "Succès", "Voyage planifié avec succès. Statut du véhicule mis à jour.")
self.load_voyages()
self.charger_comboboxes()

# Nettoyage des champs après succès
self.input_depart.clear()
```

```
        self.input_arrivee.clear()
        self.input_distance_estimee.clear() # Nettoyage du nouveau champ
        self.combo_client.setCurrentIndex(0)
        self.combo_chauffeur.setCurrentIndex(0)
        self.combo_vehicule.setCurrentIndex(0)

    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de planifier le voyage: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()

def terminer_voyage(self):
    """Clôture un voyage sélectionné, met à jour les données et remet le véhicule 'Disponible'."""
    selected_item = self.tableVoyages.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un voyage 'Planifié' à terminer.")
        return
```

```
id_voyage = selected_item.text(0)
statut_actuel = selected_item.text(4)
immatriculation = selected_item.text(3)

if statut_actuel != 'Planifié':
    QMessageBox.warning(self, "Statut Invalide", "Seuls les voyages 'Planifiés' peuvent être terminés.")
    return

try:
    conn = self.db_connect()
    cursor = conn.cursor()

    date_fin = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    # Mettre à jour le voyage
    cursor.execute("""
        UPDATE VOYAGE SET
        statut_voyage = 'Terminé',
        date_fin_reelle = ?
```

```
        WHERE id_voyage = ?
    """, (date_fin, id_voyage))

    # Mettre à jour le statut du VEHICULE à 'Disponible'
    cursor.execute("UPDATE VEHICULE SET statut_disponibilite = 'Disponible' WHERE immatriculation = ?", (immatriculation,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Voyage ID {id_voyage} terminé. Le véhicule est de nouveau disponible.")
    self.load_voyages()
    self.charger_comboboxes()

    # Émission du signal 'voyage_termine' pour synchroniser les autres modules
    self.voyage_termine.emit()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de terminer le voyage: {e}")
finally:
    if 'conn' in locals() and conn:
        conn.close()
```

```
def generer_facture(self):
    """Génère une facture pour un voyage terminé (FACTURE_CLIENT)."""
    selected_item = self.tableVoyages.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un voyage 'Terminé' à facturer.")
        return

    id_voyage = selected_item.text(0)
    statut_actuel = selected_item.text(4)

    if statut_actuel != 'Terminé':
        QMessageBox.warning(self, "Statut Invalide", "Seuls les voyages 'Terminés' peuvent être facturés.")
        return

    try:
        conn = self.db_connect()
        cursor = conn.cursor()
```

```
    # Vérifier si une facture existe déjà
    cursor.execute("SELECT id_facture FROM FACTURE_CLIENT WHERE id_voyage = ?", (id_voyage,))
    if cursor.fetchone():
        QMessageBox.information(self, "Information", f"Le voyage ID {id_voyage} a déjà été facturé.")
        return

    # Insérer la nouvelle facture (simplifié : insère juste l'ID voyage)
    cursor.execute("INSERT INTO FACTURE_CLIENT (id_voyage) VALUES (?)", (id_voyage,))

    # Mettre à jour le statut du voyage à 'Facturé'
    cursor.execute("UPDATE VOYAGE SET statut_voyage = 'Facturé' WHERE id_voyage = ?", (id_voyage,))

    conn.commit()
    QMessageBox.information(self, "Succès", f"Facture générée (ID Voyage: {id_voyage}). Statut mis à jour à 'Facturé'.")
    self.load_voyages()

except sqlite3.Error as e:
    QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de générer la facture: {e}")
finally:
```

if 'conn' in locals() and conn:

conn.close()

Absolument, l'erreur `NOT NULL constraint failed: FACTURE_CLIENT.date_emission` provient du fait que votre requête d'insertion dans `generer_facture` est incomplète :

Python
```
# Code actuel et erroné dans generer_facture :
cursor.execute("INSERT INTO FACTURE_CLIENT (id_voyage) VALUES (?)", (id_voyage,))
# Manque date_emission, montant_HT, montant_TVA, statut_paiement...
```

Voici les étapes pour corriger la méthode `generer_facture` dans `GestionVoyage.py` afin d'intégrer toutes les contraintes de la table `FACTURE_CLIENT`.

Correction de la Méthode `generer_facture` (avec Calcul)

Nous allons modifier la méthode pour :

1. Récupérer la distance estimée du voyage.
2. Calculer le `montant_HT` et le `montant_TVA`.
3. Définir la `date_emission` (date du jour) et la `date_echeance` (date du jour + 30 jours).
4. Effectuer l'insertion complète dans `FACTURE_CLIENT`.

1. Imports Nécessaires

Ajoutez l'import `timedelta` au début du fichier `GestionVoyage.py` :

Python
```
from datetime import datetime, timedelta # <- MODIFICATION
# ...
```

2. Implémentation de `generer_facture`

Remplacez entièrement votre méthode `generer_facture` actuelle par la version suivante :

Python

```python
def generer_facture(self):
    """Génère une facture pour un voyage terminé (FACTURE_CLIENT) avec calculs."""
    selected_item = self.tableVoyages.currentItem()
    if not selected_item:
        QMessageBox.warning(self, "Sélection Requise", "Veuillez sélectionner un voyage 'Terminé' à facturer.")
        return

    # Récupération des données du voyage sélectionné dans le tableau
    id_voyage = selected_item.text(0)
    statut_actuel = selected_item.text(4)

    # L'index 8 correspond à 'Distance Estimée (km)' dans votre nouvelle requête load_voyages
    distance_estimee_str = selected_item.text(8)

    # ⚠️ Validation du Statut
    if statut_actuel != 'Terminé':
        QMessageBox.warning(self, "Statut Invalide", "Seuls les voyages 'Terminés' peuvent être facturés.")
        return

    try:
        # Conversion de la distance
        distance_estimee = float(distance_estimee_str) if distance_estimee_str else 0.0

        # 🎯 DÉFINITION DES RÈGLES DE FACTURATION
        TARIF_KM_HT = 2.50  # Exemple de tarif : 2.50€ par km (Hors Taxes)
        TAUX_TVA = 0.20     # Taux de TVA (20%)

        # 🎯 CALCUL DES MONTANTS
        montant_ht = round(distance_estimee * TARIF_KM_HT, 2)
        montant_tva = round(montant_ht * TAUX_TVA, 2)

        # 🎯 CALCUL DES DATES
        date_emission_dt = datetime.now()
        date_echeance_dt = date_emission_dt + timedelta(days=30) # Échéance à 30 jours

        # Formatage pour l'insertion dans SQLite
        date_emission = date_emission_dt.strftime('%Y-%m-%d %H:%M:%S')
        date_echeance = date_echeance_dt.strftime('%Y-%m-%d %H:%M:%S')
        statut_paiement = 'En attente' # Valeur par défaut

        conn = self.db_connect()
        cursor = conn.cursor()

        # 1. Vérifier si une facture existe déjà (par id_voyage UNIQUE)
```

```
        cursor.execute("SELECT id_facture FROM FACTURE_CLIENT WHERE id_voyage = ?", (id_voyage,))
        if cursor.fetchone():
            QMessageBox.information(self, "Information", f"Le voyage ID {id_voyage} a déjà été facturé.")
            return

        # 2. Insérer la nouvelle facture avec TOUTES les colonnes NOT NULL
        cursor.execute("""
            INSERT INTO FACTURE_CLIENT (id_voyage, date_emission, date_echeance, montant_HT, montant_TVA, statut_paiement)
            VALUES (?, ?, ?, ?, ?, ?)
        """, (id_voyage, date_emission, date_echeance, montant_ht, montant_tva, statut_paiement))

        # 3. Mettre à jour le statut du voyage à 'Facturé'
        cursor.execute("UPDATE VOYAGE SET statut_voyage = 'Facturé' WHERE id_voyage = ?", (id_voyage,))

        conn.commit()
        QMessageBox.information(self, "Succès", f"Facture générée (ID Voyage: {id_voyage}). Montant HT: {montant_ht}€. Statut
mis à jour.")
        self.load_voyages()

    except ValueError:
        QMessageBox.critical(self, "Erreur de Calcul", "La distance estimée n'est pas un nombre valide.")
    except sqlite3.Error as e:
        QMessageBox.critical(self, "Erreur Base de Données", f"Impossible de générer la facture: {e}")
    finally:
        if 'conn' in locals() and conn:
            conn.close()
```

Réponse aux Questions Spécifiques

2) Comment remplir le CHAMP nommé "date_emission" ?

Le champ `date_emission` doit contenir la **date et l'heure actuelles** (`datetime.now()`) au moment où la fonction `generer_facture` est exécutée.

- **Remplissage :**

  Python

- `date_emission_dt = datetime.now()`
- `date_emission = date_emission_dt.strftime('%Y-%m-%d %H:%M:%S')`
-

- **Cohérence avec la `Date Fin Réelle` :** Non, ils ne devraient pas forcément être identiques. La **Date Fin Réelle** marque la fin du service (le voyage), tandis que la **Date d'Émission** marque l'acte administratif de création de la facture.

3) Comment également remplir tous les autres champs ?

Champ	Contenu	Logique de Remplissage
`date_echeance`	Date limite de paiement.	Calculée : `date_emission + timedelta(days=30)` (ou autre délai standard).
`montant_HT`	Coût du voyage avant taxes.	Calculé : `distance_estimee * Tarif_HT_par_Km`.
`montant_TVA`	Montant des taxes.	Calculé : `montant_HT * Taux_TVA`.
`statut_paiement`	État initial de la facture.	Valeur par défaut : `'En attente'`.

L'implémentation ci-dessus utilise la `distance_estimee` (récupérée de la colonne 8 du `QTreeWidget`) pour effectuer ces calculs.

Maintenant quand l'utilisateur clique sur le bouton "Génerer Facture" alors un message apparaît . Je voudrais savoir où se trouve le fichier de la facture généré

Quand l'utilisateur clique sur le bouton **"Générer Facture"** dans votre application, **aucun fichier de facture réel (PDF, etc.) n'est généré**.

Le terme "Facture générée" dans votre code fait référence à l'**enregistrement de la facture dans la base de données** SQLite (`bdd.db`).

Voici ce qui se passe concrètement :

---

1. Localisation de la "Facture" (Enregistrement)

Le code `generer_facture` exécute une requête SQL **`INSERT`** qui crée une nouvelle ligne dans la table **`FACTURE_CLIENT`** de votre base de données :

SQL
```
INSERT INTO FACTURE_CLIENT (...) VALUES (...)
```

- **Le fichier de la facture est :** La ligne nouvellement insérée dans la table `FACTURE_CLIENT` de la base de données **`bdd.db`**.
- **Où se trouve cette information :** Elle est stockée dans le fichier **`bdd.db`** (qui se trouve généralement dans le même répertoire que votre script `main.py`).

L'application ne produit **pas** de document externe imprimable ou visualisable.

2. Étapes pour Générer un Vrai Fichier

Si vous souhaitez que l'application génère un fichier consultable (comme un PDF ou un HTML), vous devriez ajouter des étapes supplémentaires à la fin de la fonction `generer_facture` :

1. **Récupérer les données :** Effectuer une dernière requête `SELECT` sur la table `FACTURE_CLIENT` pour obtenir l'ID de la facture et toutes les informations associées (Client, Voyage, Montants).
2. **Utiliser une librairie :** Utiliser une librairie Python dédiée à la création de documents (comme `ReportLab` pour les PDF, ou des outils de template HTML/Word) pour mettre en forme ces données.
3. **Sauvegarder le fichier :** Sauvegarder le fichier généré sur le disque de l'utilisateur.

Pour l'instant, le message de succès est uniquement une confirmation que l'enregistrement a été fait dans la base de données.

Vérifiez les réponses de Gemini, car il peut se tromper.